AIN SHAMS UNIVERSITY · Faculty of Engineering

MCT333 / MCT344 **Mechatronic Systems Design**

Spring 2026

# Mecanum-Wheeled Robot Manipulator

*Modular Mobile Manipulator for Autonomous Cube Sorting*


Group Members

| | | |
|---|---|---|
| 01 | Yousuf Eslam Mohammed | 2300605 |
| 02 | Abdelrahman Ahmed Zakaria | 2300146 |
| 03 | Tarek Salem Abdulhamid | 2300149 |
| 04 | Abdelrahman M. Abdulhamid | 2300172 |
| 05 | Ibrahim Fayez Mohammed | 2300056 |
| 06 | Mohammed Baiomy Abdelqader | 2300830 |
| 07 | Raed Hassan | 2300238 |
| 08 | Hassan Ahmed Hassan | 2300129 |


**Submission:** Spring 2026

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Project Overview

## 1.1 Objectives and Competition Requirements

The mecanum wheeled robot is an automatic sorting and transport robot developed for an academic engineering competition. The system is built around two cooperating modules: a Mecanum-wheeled mobile base capable of omnidirectional motion, and a multi-degree-of-freedom robotic arm mounted on top of the base. Together they form a single integrated platform that can navigate to a pick-up area, identify cubes, grasp them, and deposit them in the correct output location.

The core task is as follows. Five-centimetre cubic objects are placed in a defined arena. Each cube carries a QR code on one of its faces. The robot uses an onboard camera to scan the code, which encodes the cube's target colour category. Based on the decoded information, the arm places the cube into the appropriate bin carried on the base. Once the bins are loaded, the robot transports them to designated drop-off zones within the arena.

Operation is split into two modes. Certain phases of the run require fully automatic behaviour for both the base and the arm. Other phases allow manual intervention, where an operator can issue commands through a laptop-based Human Machine Interface (HMI). The HMI communicates with the robot in real time and provides status feedback to the operator. This hybrid control structure allows the team to handle edge cases during competition while still satisfying the autonomy requirements of each phase.

## 1.2 System Constraints

The competition regulations impose a set of hard design constraints that governed every major decision in this project. These are summarised in Table 1.1.

**Table 1.1.** Design constraints

| Parameter | Limit / Requirement | Notes |
|---|---|---|
| Total weight | $< 10\,\text{kg}$ | Includes motors, mechanics, electronics, and batteries |
| Max dimensions (arm folded) | $50 \times 50 \times 70\,\text{cm}$ (W×D×H) | Robot must start in this configuration |
| Cube size | $5 \times 5 \times 5\,\text{cm}$ | Each cube carries a QR code encoding its target category |
| Materials | 3D printing, laser cutting, or approved alternatives | Other methods require instructor approval |
| Power source | Battery only | Must include fuse, main switch, and safe charging provisions |
| Safety | Mandatory Phase Inspection | Must pass to enter competition |

The 10 kg weight ceiling is the tightest constraint from a design perspective. It covers the full loaded mass of the robot, including all structural parts, motors, PCBs, cabling, and battery packs. This drove the use of lightweight materials throughout: 3D-printed PLA and PETG for structural components, aluminium extrusions for the frame, and a compact lithium battery pack positioned low in the chassis.

The folded envelope of $50 \times 50 \times 70\,\text{cm}$ defines the starting configuration, meaning the arm must retract to fit within this boundary before any run begins. This shaped the arm's kinematic design and the choice of link lengths.

All power must come from onboard batteries. No tethered supply is permitted during a run. The power system therefore includes a dedicated fuse, a main power switch accessible from outside the chassis, and circuitry for safe battery charging in the pit area.

## 1.3 Team Structure and Responsibilities

The project was carried out by a student team divided across four main work streams, each aligned with a subsystem of the robot. Table 1.2 lists the primary areas of responsibility.

**Table 1.2.** Team responsibilities by subsystem.

| Work Stream | Scope |
|---|---|
| Mechanical Design | Mobile base frame, Mecanum wheel assembly, robotic arm linkages, gripper, and all 3D-printed structural parts |
| Electronics and Power | PCB design, motor driver selection, power distribution, fusing, and battery management |
| Firmware and Control | ESP32 firmware for base and arm, stepper and servo control, sensor interfacing, and inter-board communication |
| Vision and HMI | Camera integration, QR code decoding, colour classification, Raspberry Pi coordination, and laptop HMI application |

Each work stream operated with a designated lead responsible for design decisions, documentation, and integration testing for that subsystem. Weekly team meetings were used to synchronise progress and resolve cross-subsystem dependencies, particularly between firmware and mechanical tolerances on the arm, and between vision output and base navigation logic.

The overall system architecture follows a layered control model: a Raspberry Pi 4 acts as the central coordinator, receiving vision data and HMI commands and dispatching motion instructions to two ESP32 microcontrollers, one dedicated to the base and one to the arm. This split is described in detail in Chapter **??**.

# Chapter 2

# System-Level Design

## 2.1 Overview

With the system requirements and concept defined in the preceding phase, the team transitioned into the system-level design stage. At this point, the project was divided across three parallel workstreams. The mechanical, electrical, and actuation subteams began translating the concept into physical form: structural geometry, PCB layouts, and motor selection. Simultaneously, the control theory, odometry, and software subteams started their work in simulation, developing the algorithms and motion models that would eventually run on the physical platform. This parallel structure compressed the development timeline significantly and meant that by the time hardware was ready for integration, the software side already had validated models to deploy against.

The overall design methodology followed the VDI 2206 macro-cycle structure: each domain progressed through its own internal design-test-revise loop, while shared integration milestones acted as synchronization points between subteams. The following sections describe how each domain iterated and, more importantly, how those iterations intersected. Figure 2.1 illustrates the V-diagram structure used throughout this phase.

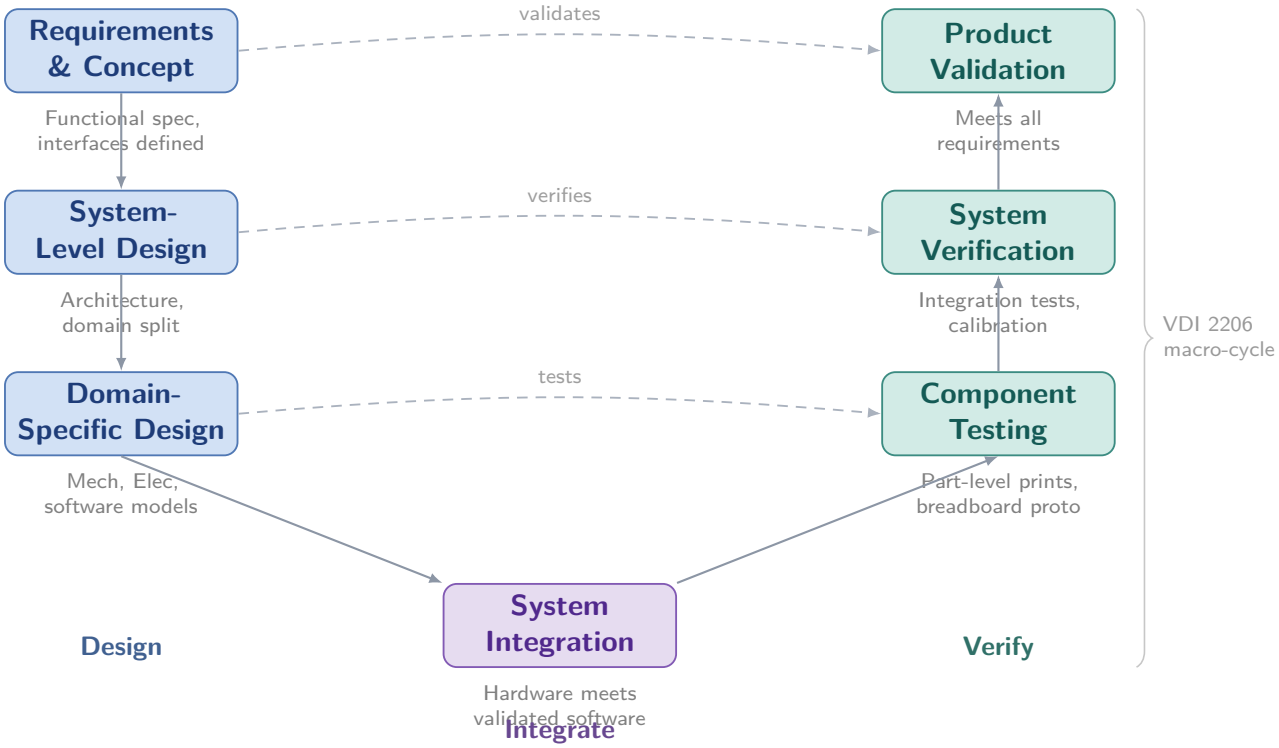


**Figure 2.1.** VDI 2206 V-diagram as applied in this project. The left branch follows system decomposition from requirements down to domain-specific implementation; the right branch mirrors it with corresponding verification and validation activities. Dashed horizontal arrows indicate which right-side activity addresses each left-side specification. System integration at the bottom is where hardware and validated software converge.

### 2.1.1 Mechanical Design Iterations

The mechanical design went through four complete CAD revisions from initial sketch to final geometry. Each revision was not simply a visual update; it reflected a resolved constraint, a fit issue discovered during test printing, or a requirement handed over from another subteam.

Rather than committing to a full print at each revision, the team adopted a component-first printing strategy. Individual parts were printed and physically tested before the full assembly was committed to manufacturing. This allowed tolerances, clearances, and mechanical concepts such as joint motion and structural load paths to be verified incrementally. Problems discovered at the part level were far cheaper to resolve than those discovered after a complete assembly.

A core principle throughout the mechanical design was the use of standardized fasteners: every structural connection in the robot relies on nuts and bolts rather than snap fits or adhesive joints. This was a deliberate modularity decision. Any single component can be removed, replaced, or reprinted without disturbing the rest of the assembly. In practice, this meant the team could iterate on a problematic part, swap it into the existing structure in minutes, and continue testing. The full prototype was rebuilt multiple times in this way, each time with only the changed component reprinted. A detailed breakdown of the fastener strategy and assembly hierarchy is presented in Section **??**.

### 2.1.2 Electrical Design Iterations

The electrical subteam followed a breadboard-first protocol for every circuit before any PCB was sent to fabrication. All signal routing, power distribution logic, and driver configurations were prototyped and validated on breadboards under realistic load conditions. This stage surfaced several issues, including noise coupling between motor driver lines and sensor signals, as well as minor timing inconsistencies in the inter-board communication protocol, both of which were corrected before the final PCB layouts were finalized.

The result was that the fabricated PCBs, covering the base controller board and the arm controller board, performed reliably from first power-on with only minor software-side tuning required. The cost of the breadboard phase was a few additional days; the benefit was avoiding a PCB respin cycle under competition timeline pressure.

## 2.2 Integration Highlights

The most consequential design decisions in this phase were not made within a single domain. They were made at the boundaries between domains, where one subteam's output became another subteam's constraint.

### Mechanical and Electrical Co-Layout

The internal volume of the robot leaves essentially no unused space. Every enclosure slot, cable routing channel, and component mounting surface was co-designed by the mechanical and electrical subteams working together. Electrical component dimensions and connector clearances were known before the final CAD revision was frozen, which meant that by the time manufacturing began, every PCB, battery pack, and wiring harness had a defined physical home in the assembly. This eliminated the common failure mode of discovering spatial conflicts during final integration.

### Actuator Sizing and CAD Integration

Motor selection did not happen in isolation from the physical design. An initial CAD sketch was used as the geometric reference during the actuator sizing process, allowing the team to reason about torque requirements, mounting constraints, and center-of-mass implications simultaneously. Once motors were selected, their exact dimensions and output shaft geometry were fed back into the CAD model, and the final motor slots, hubs, and mounting interfaces were designed around the confirmed hardware. This closed the loop between the actuation and mechanical domains cleanly, with no geometric surprises during assembly.

### Perception and Mechanical Placement

The placement of the vision camera was determined jointly by the mechanical and perception subteams. Camera position affects both the physical structure, since it requires a rigid, vibration-isolated mount at a specific location on the frame, and the software pipeline, since field of view, working distance, and occlusion geometry all influence how reliably the sorting algorithm performs. Rather than placing the camera based on mechanical convenience and hoping the perception team could work with the result, both subteams iterated on the mount position together until a placement was found that satisfied structural, geometric, and optical requirements simultaneously.

### 2.2.1 Simulation and Physical Development in Parallel

While the hardware subteams were iterating through the processes described above, the software, control theory, and odometry subteams were developing and validating their respective models in simulation. The kinematic model of the arm, the Mecanum wheel odometry equations, and the high-level task sequencing logic were all functional before the physical robot was complete. This meant that the integration phase, where software meets hardware for the first time, was a process of calibration and tuning rather than fundamental debugging. The parallel development structure, enabled by the early agreement on interfaces and coordinate conventions between subteams, was one of the more consequential process decisions made during this phase.

# Chapter 3

# Design Philosophy and Methodology

The top-level mission is decomposed systematically following VDI 2206, breaking the autonomous sorting task down from a single high-level objective into the discrete sub-functions implemented across the Raspberry Pi and the two ESP32 controllers. Three flow types are tracked throughout this decomposition and are colour-coded consistently across every diagram and table in this chapter: ≡ orange for energy, • green for material, and ∼ purple for signal. Table 3.1 summarises this convention before it is applied to the function decomposition, the flow analysis, and the functional block diagrams that follow.

## 3.1 Flow Legend

**Table 3.1.** Flow Legend

| Flow Type | Description |
|---|---|
| ≡ ENERGY FLOW | Electrical or mechanical power transfer |
| • MATERIAL FLOW | Physical objects (cubes) or consumables |
| ∼ SIGNAL FLOW | Commands, sensor data, or control information |

## 3.2 Top-Level Function Decomposition

The mission — sorting and delivering colour-coded cubes autonomously — is broken down across three levels. L0 is the overall mission; L1 captures the five major sub-systems required to achieve it (navigation, perception, manipulation, delivery, and pose estimation); and L2 lists the concrete operations performed within each sub-system. Table 3.2 presents this decomposition, which forms the basis for the module boundaries used later in the functional block diagrams.

**Table 3.2.** Top-Level Function Decomposition

| Level | Function | Sub-Functions |
|---|---|---|
| L0 | Sort and Deliver Colour-Coded Cubes Autonomously | L1 decomposition below |
| L1.1 | Navigate Arena (manual + autonomous) | L2: Drive holonomically, Stabilise heading, Detect obstacles |
| L1.2 | Detect and Identify Cube | L2: Capture frame, Decode QR code, Classify colour |
| L1.3 | Pick and Store Cube | L2: Compute IK, Grasp cube, Retract arm, Sort into bin |
| L1.4 | Deliver to Drop Zone | L2: Navigate to zone, Execute constrained manoeuvre, Release cube |
| L1.5 | Estimate Robot Pose (Odometry) | L2: Read encoders, Integrate IMU gyro, Fuse with complementary filter / EKF |

## 3.3 Complete Flow Analysis

With the function structure established, every energy, material, and signal path between sub-systems is traced explicitly. Power flows from the LiPo battery through the distribution board to the drive motors, servos, and logic rails; cubes move physically from the pedestal through the gripper, into the on-board colour bin, and out to the drop zone; and signals carry joystick commands, UART instructions, encoder ticks, IMU readings, and watchdog heartbeats between the Raspberry Pi and the two ESP32 controllers. Table 3.3 lists each of these flows together with its source, destination, and a short description, providing the same level of traceability that the embedded firmware design relies on when interpreting UART commands.

**Table 3.3.** Complete Flow Analysis — Energy / Material / Signal

| Flow Type | Source | Destination | Description |
|---|---|---|---|
| ≡ ENERGY | LiPo 4S Battery (14.8 V) | Power distribution board | Main bus 14.8 V DC; fused per rail |
| ≡ ENERGY | Power distribution board | 4× DC motors + motor drivers | 12 V traction power for mecanum drive |
| ≡ ENERGY | Power distribution board | Servo motors ×4 (arm + gripper) | 5–7.4 V servo power rail |
| ≡ ENERGY | Power distribution board | Raspberry Pi 4 + both ESP32s | 5 V regulated logic rail (3 A each) |
| ≡ ENERGY | DC motors ×4 | Mecanum wheels → arena floor | Mechanical torque → holonomic motion (X/Y/$\theta$) |
| ≡ ENERGY | Servo motors ×3 | Arm joints 1–3 | Joint torque for arm trajectory |
| ≡ ENERGY | Gripper servo | Parallel gripper fingers | Clamping force on 5×5×5 cm cube |
| • MATERIAL | Pickup pedestal (40 cm ht.) | Robot arm gripper | 5×5×5 cm QR-coded cube grasped from pedestal |
| • MATERIAL | Gripper / arm | On-board colour bin (R/G/B) | Cube stored in correct bin during transit |
| • MATERIAL | On-board colour bin | Drop-off zone floor | Cube released inside matching colour square |
| ~ SIGNAL | Operator wireless controller (2.4 GHz) | Raspberry Pi 4 | Joystick Vx, Vy, $\omega$ setpoints — teleop phase |
| ~ SIGNAL | Raspberry Pi 4 | ESP32 Base (UART) | Motion commands: BASE:VEL / BASE:TURN / BASE:STOP |
| ~ SIGNAL | Raspberry Pi 4 | ESP32 Arm (UART) | Arm commands: ARM:MOVE / ARM:GRIP:OPEN / ARM:GRIP:CLOSE |
| ~ SIGNAL | Wheel encoders ×4 (PCNT) | ESP32 Base | Tick counts → d_left, d_right for odometry |

**Table 3.3 – continued from previous page**

| Flow Type | Source | Destination | Description |
|---|---|---|---|
| ~ SIGNAL | ESP32 Base | Raspberry Pi 4 (UART) | Telemetry: BASE:ENC:120,118 / BASE:IMU:24.5 |
| ~ SIGNAL | IMU — MPU-6050 (I2C) | ESP32 Base | $\omega_z$ (gyro) + accel → complementary filter → $\theta_{final}$ |
| ~ SIGNAL | Ultrasonic sensors ×2 | ESP32 Base | Obstacle range: BASE:ULTRA:Front=35 |
| ~ SIGNAL | RPi Camera Module V2 (IMX219 8 MP, CSI-2) | Raspberry Pi 4 | Frame stream → QR decode (pyzbar) + cube pose (OpenCV) |
| ~ SIGNAL | Raspberry Pi 4 (perception) | Raspberry Pi 4 (state machine) | Colour ID + cube pose → pick trigger |
| ~ SIGNAL | Arm joint encoders / limit switches | ESP32 Arm | Joint angle feedback + hard-limit enforcement |
| ~ SIGNAL | ESP32 Arm | Raspberry Pi 4 (UART) | ARM:POS:90 / ARM:FORCE:2.4 / ARM:OBJECT:1 |
| ~ SIGNAL | E-Stop button (latching) | Both ESP32s via RPi | Immediate halt broadcast on press |
| ~ SIGNAL | Software watchdog (RPi) | Both ESP32s (UART) | Heartbeat timeout → BASE:STOP + ARM:STOP |

## 3.4 Functional Block Diagrams

The flows traced above are now grouped by module, mirroring the breakdown used in the embedded firmware (Chapter 8):The system is composed of an HMI/teleoperation layer, a Raspberry Pi–based integration layer, a perception module, a mobile base, and a manipulator. Each functional block diagram defines its inputs, internal processing, and outputs using a consistent energy/material/signal color convention, ensuring clear separation between hardware modules and the commands exchanged between them.

## System-Level Functional Block Diagram

figure 3.1 gives the system-wide view: the HMI layer issues joystick and mode-toggle signals; the integration layer on the Raspberry Pi runs the state machine and routes UART traffic to both ESP32s; the perception module turns camera frames into colour IDs and cube poses; and the mobile base and manipulator modules consume energy and signal commands to produce physical motion.

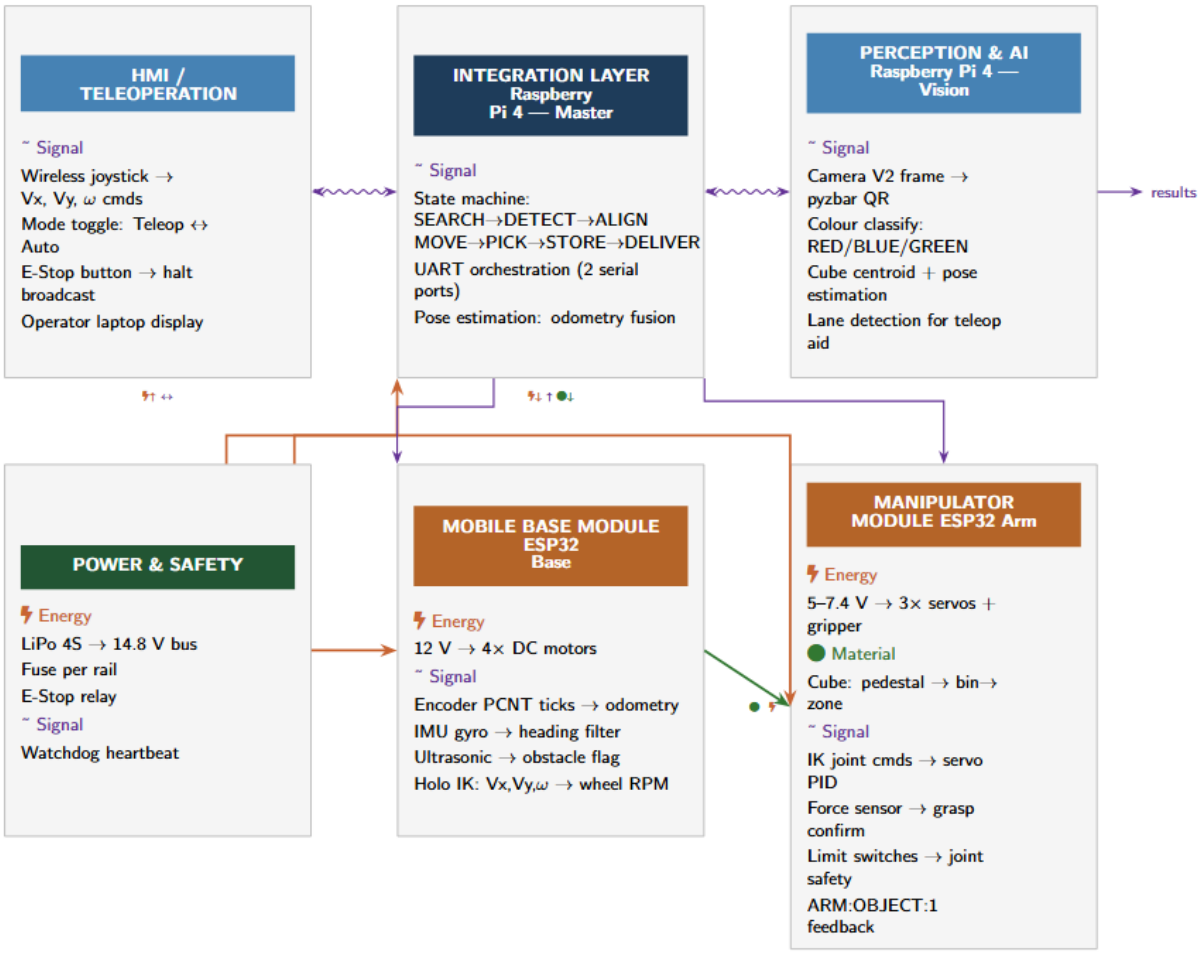


**Figure 3.1.** System-Level Functional Block Diagram (FBD)

## Mobile Base Module FBD

The Base ESP32 (described in detail in Section 8.3) consumes 12 V traction power and Vx/Vy/$\omega$ velocity commands from the Pi, converts them through the mecanum inverse-kinematics equations into per-wheel PWM duty cycles, and feeds encoder ticks and IMU gyro readings back into the heading filter. Table 3.4 details these inputs, processes, and outputs, including the 600 ms watchdog cutoff that halts the motors if commands stop arriving.

**Table 3.4.** Mobile Base Module FBD

| Flow Type | Input | Process | Output |
|---|---|---|---|
| ≡ Energy | 12 V from power bus | Motor drivers (BTS7960) → PWM | Wheel torque → holonomic motion |
| ~ Signal | Vx, Vy, $\omega$ from RPi (UART) | Mecanum IK: $\omega_1 = V_x - V_y - \omega L$, $\omega_2 = V_x + V_y + \omega L$ … | PWM duty cycles to 4 drivers |
| ~ Signal | Encoder PCNT ticks ×4 | $d_{\text{left}} = \text{ticks/CPR} \times \pi \times D$; $d_{\text{center}} = (d_R + d_L)/2$ | X,Y,$\theta$ via dead-reckoning (10 ms loop) |
| ~ Signal | IMU gyro $\omega_z$ (MPU-6050 I2C) | $\theta_{\text{gyro}} = \theta_{\text{old}} + (\omega \times \Delta t) -$ bias | $\theta_{\text{final}} = 0.98 \times \theta_{\text{gyro}} + 0.02 \times \Delta\theta_{\text{enc}}$ |
| ~ Signal | Ultrasonic echo (×2 front) | Distance compare → obstacle flag | BASE:ULTRA:Front=35 → RPi |
| ~ Signal | BASE:STOP from RPi watchdog | Immediate PWM cut (all channels) | Motors halt within 50 ms |

## Manipulator Module FBD

The Arm ESP32 takes ARM:MOVE and ARM:GRIP commands from the Pi, resolves them through the on-board IK solver into joint setpoints for the servo PID loops described in Section 8.3, and physically carries the cube from the pedestal through the gripper into the on-board bin and out to the drop zone. Table 3.5 lists each stage of this energy, material, and signal flow, including the limit-switch safety path.

## Perception Module FBD

Running on the Raspberry Pi's vision pipeline, the perception module reads camera frames at up to 1080p30, decodes the QR code with pyzbar in under 5 ms, and derives both a colour ID and a 3D cube pose from the OpenCV contour analysis. Table 3.6 also includes the lane-detection process that assists the operator during teleoperation.

## HMI & Teleoperation Module FBD

The same PS4, keyboard, and dashboard interfaces introduced in Section 8.4 are represented here purely in terms of the signals they generate: joystick axes become Vx/Vy/$\omega$ velocity commands, the mode-toggle button drives the state-machine transition, and the latching E-Stop broadcasts an immediate halt to both ESP32s through the Pi. Table 3.7 also shows the telemetry path that prints encoder and IMU data on the operator's terminal, as

**Table 3.5.** Manipulator Module FBD

| Flow Type | Input | Process | Output |
|---|---|---|---|
| ≡ Energy | 5–7.4 V servo rail | Servo PWM pulse → motor winding | Joint torque → arm motion |
| • Material | Cube on 40 cm pedestal | Gripper closes around cube | Cube secured in jaws |
| • Material | Cube in gripper | Arm retracts → positions over bin | Cube deposited in colour bin |
| • Material | Cube in on-board bin | Arm extends over drop zone → open | Cube released in correct zone |
| ~ Signal | ARM:MOVE:⟨$\theta$⟩ from RPi | IK → joint angle setpoints → servo PID | ARM:POS:⟨$\theta$⟩ feedback to RPi |
| ~ Signal | ARM:GRIP:CLOSI from RPi | Gripper servo PWM → close | ARM:FORCE:2.4 + ARM:OBJECT:1 to RPi |
| ~ Signal | Limit switch inputs | Hard-limit enforcement firmware | ARM:LIMIT:1 alert + servo halt |

**Table 3.6.** Perception Module FBD

| Flow Type | Input | Process | Output |
|---|---|---|---|
| ≡ Energy | 5 V via CSI-2 ribbon | IMX219 sensor power | Camera operational |
| ~ Signal | Camera V2 frame (720p60 / 1080p30) | pyzbar QR decode (< 5 ms/frame) | Colour ID string: RED / BLUE / GREEN |
| ~ Signal | Camera V2 frame | OpenCV contour → centroid → pixel error | 3D cube pose (x, y, z) relative to base |
| ~ Signal | Colour ID | Drop-zone lookup table | Target zone coordinates (x, y, $\theta$) |
| ~ Signal | Frame — lane markings | HSV threshold + Hough lines | Lateral tracking error for teleop guidance |

required by the competition rules.

**Table 3.7.** HMI & Teleoperation Module FBD

| Flow Type | Input | Process | Output |
|---|---|---|---|
| ~ Signal | 2.4 GHz joystick axes | Deadband + scaling → velocity commands | Vx, Vy, $\omega$ → RPi → UART → ESP32 Base |
| ~ Signal | Mode toggle button | State machine transition trigger | Teleop / Autonomous flag to RPi |
| ~ Signal | E-Stop button (latching, NC) | Immediate halt broadcast | BASE:STOP + ARM:STOP via RPi |
| ~ Signal | ESP32 Base UART telemetry | Parse & display: encoder ticks + IMU yaw | Printed on laptop terminal (required by rules) |
| ~ Signal | Perception colour result | Display colour ID + target drop zone | Visual confirmation on operator screen |

## 3.5 TRIZ Contradiction Analysis

TRIZ is applied to six key engineering contradictions that arose during the mechanical and control design of the robot. Each contradiction is mapped to the 39-parameter Altshuller Contradiction Matrix to extract candidate Inventive Principles, which are then interpreted in the context of this specific design.

### Contradiction Statements

Table 3.8 lists each contradiction as an improving parameter traded off against a worsening one — for example, increasing arm reach and payload (C1) tends to increase the robot's stowed footprint, while improving positioning accuracy (C2) tends to demand a heavier battery.

### TRIZ Contradiction Matrix — Project Extract

Rows represent the improving parameter and columns the worsening parameter in Table 3.9. The starred cell [7 × 3] gives principles 1, 7, 4, 35, matching the red-boxed reference cell in the Altshuller matrix image provided, and resolves the arm-volume-versus-footprint contradiction (C1).

**Table 3.8.** TRIZ Contradiction Statements

| # | Improving Parameter | Worsening Parameter | Altshuller Parameters | Inventive Principles |
|---|---|---|---|---|
| C1 | Arm reach / payload (Volume of moving object — Para 7) | Robot footprint / stowed size (Length of moving object — Para 3) | 7 × 3 | 1, 7, 4, 35 |
| C2 | Positioning accuracy (Speed — Para 9) | Battery capacity (Weight of moving object — Para 1) | 9 × 1 | 28, 13, 35 |
| C3 | Arm structural rigidity (Strength — Para 14) | Arm link mass (Weight of moving object — Para 1) | 14 × 1 | 1, 8, 15, 40 |
| C4 | Gripper reliability (Reliability — Para 27) | Gripper complexity (Weight of moving object — Para 1) | 27 × 1 | 11, 28, 35 |
| C5 | QR detection speed (Speed — Para 9) | System ease of use (Ease of operation — Para 33) | 9 × 33 | 10, 13, 28 |
| C6 | Omni-wheel traction (Force — Para 10) | Robot footprint (Length of moving object — Para 3) | 10 × 3 | 1, 7, 4, 35 |

**Table 3.9.** TRIZ Contradiction Matrix — Project Extract

| Improving ↓ / Worsening → | 1. Weight | 3. Length | 9. Speed | 14. Strength | 27. Reliab. | 33. Ease of use |
|---|---|---|---|---|---|---|
| **1. Weight moving** | * | 8,15 29,34 | 2,8 15,38 | 1,8 15,40 | 11,28 3,35 | — |
| **3. Length moving (C6→)** | 8,15 29,34 | * | 13,4 8,10 | — | — | — |
| **7. Volume moving (C1→)** | 2,26 29,40 | **1,7 4,35 ★** | 15,7 4,35 | 7,17 4,35 | 35,34 2,10 | — |
| **9. Speed (C2,C5→)** | 2,28 13,38 | 13,4 8,10 | * | 9,14 15,35 | 21,35 28,10 | **10,13 28** |
| **10. Force (C6→)** | 1,7 4,35 | — | 9,14 15,35 | — | — | — |
| **14. Strength (C3→)** | 1,8 15,40 | 1,15 8,35 | 10,9 14,35 | * | 11,28 10,35 | — |
| **27. Reliability (C4→)** | 11,28 3,35 | 11,35 3,28 | 21,35 28,10 | 11,28 10,35 | * | — |

## Inventive Principles

The principles extracted from the matrix were translated into concrete design choices. Segmentation and nesting shaped the collapsible, telescoping arm; asymmetry and counterweighting governed the placement of the battery relative to the arm mount; dynamics and composite materials drove the choice of carbon-fibre-reinforced PLA for the arm links; and mechanics substitution and parameter change resolved the trade-offs in transmission design and battery chemistry. Table 3.10 summarises how each principle was applied.

**Table 3.10.** Inventive Principles Applied

| Principle | Name | Application to This Robot |
|---|---|---|
| 1 | Segmentation | Divide arm into 3 modular, collapsible links — folds within 70 cm height limit; extends to reach 40 cm pedestal. |
| 4 | Asymmetry | Offset battery opposite the arm mount point to counterbalance arm mass without extra structural weight. |
| 7 | Nesting | Telescope arm link 2 inside link 1 when folded — maximum reach-to-stowed-size ratio. |
| 8 | Counterweight | Mount 4S LiPo opposite arm; reduces effective tipping torque on mecanum base. |
| 10 | Preliminary action | Pre-compute arm IK joint angles on RPi before robot stops at pedestal — eliminates planning latency on arrival. |
| 11 | Beforehand cushioning | TPU compliant fingertips on gripper — absorbs ±2 mm positional error; improves grasp reliability. |
| 13 | Other way round | Instead of navigating robot precisely to cube, use camera centroid error to micro-correct with Pure-X/Y approach. |
| 15 | Dynamics | CFRPLA lattice links — high specific stiffness, low mass; load-optimised dynamic structure for arm. |
| 28 | Mechanics substitution | Belt-pulley arm joint transmission instead of gearbox — 35% lighter per joint, same output torque. |
| 35 | Parameter change | Switch NiMH → 4S LiPo: same voltage, 40% lighter per Wh — resolves C2 (accuracy vs. battery weight). |
| 40 | Composite materials | Carbon-fibre-reinforced PLA arm links — high specific stiffness; directly resolves C3 (strength vs. weight). |

# Chapter 5

# Actuator Sizing and Selection

The selection of suitable actuators for the 4-DOF robotic manipulator was performed through dynamic analysis using MATLAB Simulink and Simscape Multibody. A complete three-dimensional model of the robotic arm was developed to accurately represent the geometry, mass distribution, and kinematic structure of the system. This model enabled the estimation of realistic torque requirements at each joint during operation.

## 5.1 Dynamic Analysis Methodology

To evaluate the actuator demands, a trapezoidal velocity profile was applied to each revolute joint. The profile consists of three phases: acceleration, constant velocity, and deceleration. This motion trajectory was selected because it closely resembles the motion characteristics of practical robotic systems while avoiding unrealistic instantaneous changes in velocity.

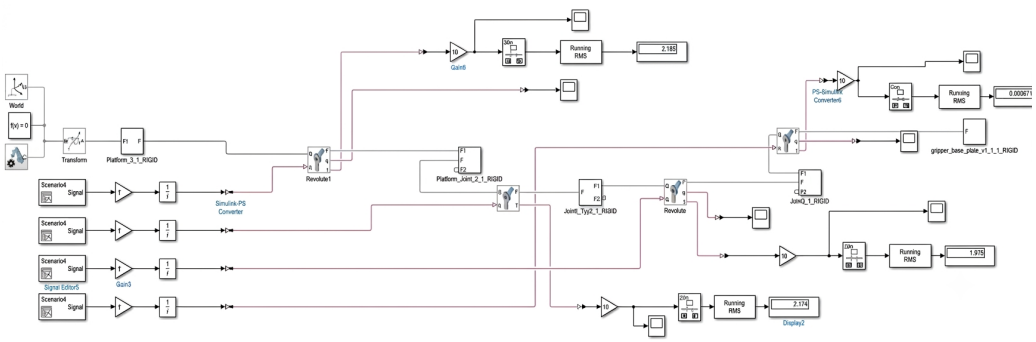


**Figure 5.1.** Dynamic model of the robotic manipulator developed using MATLAB Simulink Simscape Multibody.

The desired joint trajectories were imposed using controlled motion sources within the Simscape environment. These blocks convert the prescribed motion profiles into joint movements, allowing the simulation to compute the corresponding reaction torques required at each joint throughout the motion cycle.

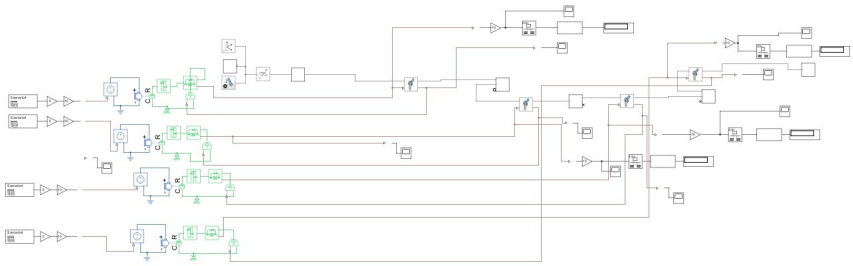

**Figure 5.2.** Controlled motion source used to impose the desired joint trajectory.

The resulting torque profiles were obtained for all revolute joints. From these profiles, the peak torque values were extracted and used as the primary design criterion for actuator selection. The simulated peak torque requirements are summarized in Table 5.1.

**Table 5.1.** Simulated Peak Torque Requirements

| Joint | Peak Torque (kg-cm) |
|---|---|
| Base | 6.43 |
| Shoulder | 6.10 |
| Elbow | 3.71 |
| Wrist | 0.91 |

Figures 5.3 and 5.4 illustrate representative torque responses obtained from the simulation for the first and second revolute joints.

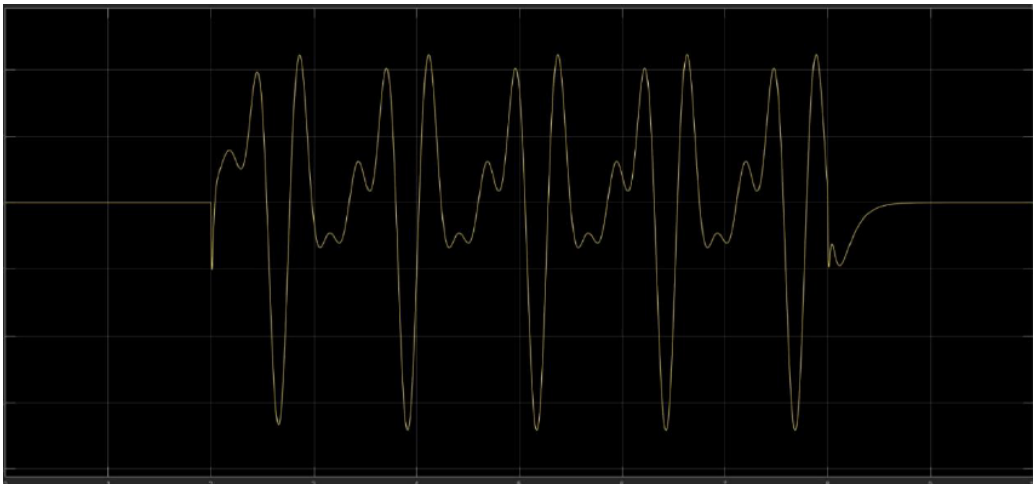

**Figure 5.3.** Torque profile of the first revolute joint during the prescribed motion cycle.

**Figure 5.4.** Torque profile of the second revolute joint during the prescribed motion cycle.

## 5.2 Actuator Selection Criteria

Since simulation models rely on simplifying assumptions such as ideal rigid bodies, negligible backlash, and limited friction effects, a safety factor was incorporated into the actuator selection process. This margin accounts for modeling uncertainties, manufacturing tolerances, payload variations, and unexpected operating conditions.

A safety factor of 1.5 was selected. Therefore, the minimum required actuator torque for each joint was calculated according to

$$T_{\text{required}} = SF \times T_{\text{simulated}} \tag{5.1}$$

where:

- $T_{\text{required}}$ is the minimum motor stall torque required,
- $T_{\text{simulated}}$ is the peak torque obtained from simulation,
- $SF = 1.5$ is the selected safety factor.

The resulting actuator requirements are presented in Table 5.2.

## 5.3 Final Actuator Configuration

After calculating the required torque values, commercially available servo motors were evaluated against the design requirements. The MG995 servo motor was selected for the

**Table 5.2.** Actuator Selection Verification

| Joint | Simulated Torque (kg-cm) | Required Torque (kg-cm) | Selected Motor |
|---|---|---|---|
| Base | 6.43 | 9.645 | MG995 |
| Shoulder | 6.10 | 9.15 | MG995 |
| Elbow | 3.71 | 5.565 | MG995 |
| Wrist | 0.91 | 1.365 | SG90 |
| Gripper | N/A | Based on gripping force requirement | SG90 |

Base, Shoulder, and Elbow joints due to their relatively high torque demands. These joints are responsible for supporting the majority of the arm's mass and payload, making high-torque actuators essential for reliable operation.

The Wrist and Gripper joints experience significantly lower loading conditions. Consequently, SG90 micro servos were selected for these joints. Their compact size, low weight, and sufficient torque capacity make them suitable for the wrist rotation and gripping mechanism while minimizing the overall mass of the manipulator.

The specifications of the selected actuators are summarized in Table 5.3.

**Table 5.3.** Selected Servo Motor Specifications

| Parameter | MG995 | SG90 |
|---|---|---|
| Operating Voltage | 4.8–7.2 V | 4.8–6 V |
| Stall Torque | 10 kg-cm | 1.8 kg-cm |
| Weight | 55 g | 9 g |
| Speed (at 4.8 V) | 0.20 s/60° | 0.12 s/60° |

As shown in Table 5.3, the selected actuators provide torque ratings greater than the minimum required values obtained from the dynamic analysis. Therefore, all joints satisfy the actuator sizing criteria with the specified safety margin.

The final actuator configuration consists of:

- Three MG995 servo motors for the Base, Shoulder, and Elbow joints.
- Two SG90 servo motors for the Wrist joint and Gripper mechanism.

## 5.4 Summary

This actuator configuration satisfies all torque requirements while maintaining a balance between performance, weight, power consumption, and overall system cost. The selected servos provide sufficient torque margins to ensure reliable operation of the robotic manipulator without motor stall or mechanical failure under the expected loading conditions.

Key design decisions and their rationale are summarised below:

**Table 5.4.** Actuator design decisions and rationale

| Decision | Rationale |
|---|---|
| MG995 for Base, Shoulder, Elbow | High torque capacity (10 kg-cm) meets safety factor requirements for heavy-load joints |
| SG90 for Wrist and Gripper | Compact size and low weight reduce arm inertia; sufficient torque for light-load joints |
| Safety factor of 1.5 | Accounts for modeling uncertainties, friction, manufacturing tolerances, and payload variations |
| Trapezoidal velocity profile | Realistic motion representation avoids instantaneous velocity changes while capturing peak torque demands |
| Simscape Multibody modeling | Enables accurate estimation of dynamic torques without physical prototyping |
| Commercial servo selection | Reduces development time and cost while ensuring reliable, off-the-shelf availability |

# Chapter 6

# Electrical Design

The electronic architecture of the robot is organised into two independent custom PCBs: an **Arm Control PCB** and a **Base Control PCB**. Separating the subsystems at the board level reduces wiring complexity, improves fault isolation, and allows each subsystem to be developed, tested, and replaced independently without affecting the rest of the robot.

## 6.1 System Architecture

Three processing units form the control hierarchy: a Raspberry Pi 4 acting as the high-level coordinator, an ESP32-based Arm Control PCB, and an ESP32-S3-based Base Control PCB. The Raspberry Pi handles all decision-making, vision processing, and coordination; the two embedded boards execute low-level actuator commands.

### Communication Architecture

All inter-board communication is routed through the Raspberry Pi. Neither the Arm PCB nor the Base PCB communicate directly with each other; this star topology simplifies synchronisation and keeps the embedded firmware straightforward.

**Table 6.1.** Communication links summary

| Communication Link | Interface |
|---|---|
| Raspberry Pi ↔ Arm PCB (ESP32) | USB Serial (UART) |
| Raspberry Pi ↔ Base PCB (ESP32-S3) | USB Serial (UART) |
| PS4 Controller ↔ Raspberry Pi | Bluetooth |
| Raspberry Pi ↔ Laptop Dashboard | Wi-Fi |

The PS4 controller connects over Bluetooth and is used for manual teleoperation of both the arm and the base. A Wi-Fi link to the laptop dashboard provides real-time monitoring and status visualisation during operation.

## 6.2 Arm Control PCB

### Overview

The Arm Control PCB is built around an ESP32 (38-pin) microcontroller. It drives five servo motors corresponding to the arm joints and forwards commands received from the Raspberry Pi via USB serial.

The arm uses three MG996R high-torque servos for the heavier joints and two SG90 micro servos for the wrist rotation and gripper. Because MG996R motors can draw substantial transient currents, the power architecture explicitly separates motor supply from logic supply.

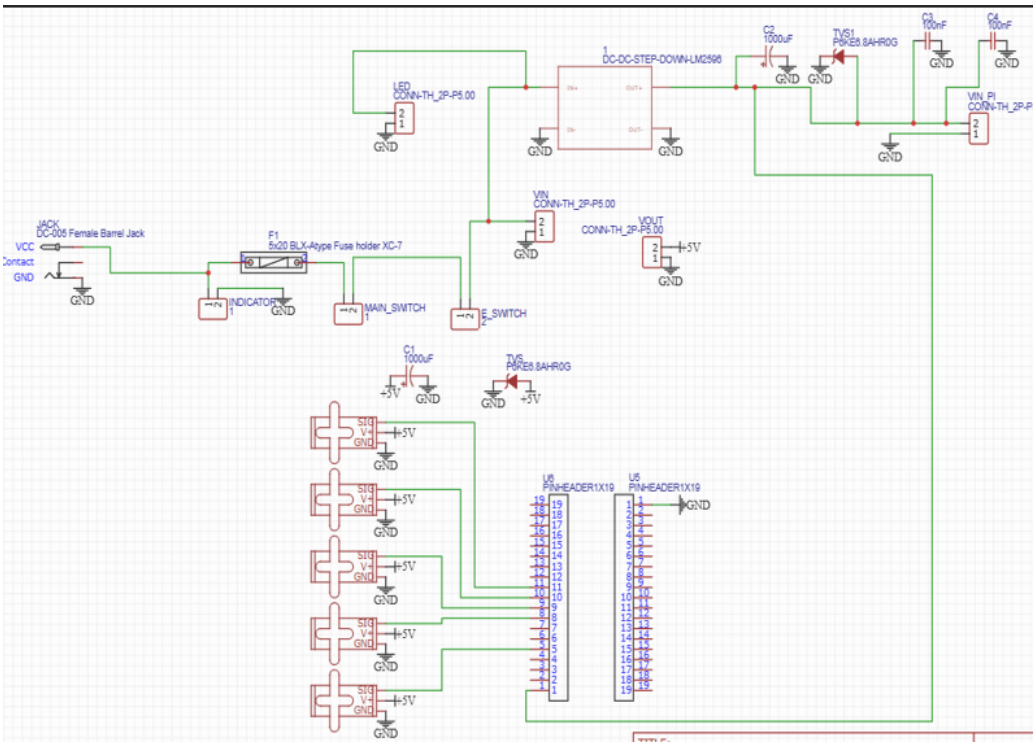


**Figure 6.1.** Arm Control PCB schematic diagram.

## Power Management

The board is supplied from a 12 V source, which is conditioned through two independent buck converters:

- **XL4016 Buck Converter:** Supplies the servo motors. Selected for its high current-handling capability, it provides a stable regulated voltage to all five servos under varying load conditions.
- **LM2596 Buck Converter:** Supplies the ESP32 and the Raspberry Pi. Isolating logic power from motor power reduces electrical noise induced by servo switching currents.

The 12 V input enters through a DC jack, passes through a fuse, and then diverges into these two independent power rails. This keeps motor load transients off the logic rail and improves overall system stability.

## PCB Design Considerations

The Arm PCB was produced as a **single-layer board** to reduce manufacturing cost and complexity.

**Trace widths** were sized to the expected current:

- Main 12 V power traces: **3 mm**
- Regulated 5 V logic traces: **2.5 mm**

**Protection circuitry** included on the board:

- TVS diodes on both buck converter output lines to suppress voltage transients.
- 1000 µF electrolytic capacitors at both converter outputs to absorb sudden current fluctuations during servo actuation.
- 100 nF ceramic decoupling capacitors placed close to the ESP32 and Raspberry Pi power pins to filter high-frequency switching noise.

Servo control signals are generated directly by the ESP32 PWM outputs; no intermediate driver circuits are required, simplifying the hardware while keeping the servo interfaces within the ESP32's drive capability.

## Hardware Components

**Table 6.2.** Arm Control PCB component list

| Component | Function |
|---|---|
| ESP32 (38-pin) | Main controller for the robotic arm |
| XL4016 Buck Converter | High-current supply for servo motors |
| LM2596 Buck Converter | Regulated supply for ESP32 and Raspberry Pi |
| 3 × MG996R Servo | High-torque arm joints |
| 2 × SG90 Servo | Wrist rotation and gripper actuation |
| 7 × 2-Way Terminal Blocks | External power and signal connections |
| DC Jack (12 V Input) | Main power input |
| Battery Indicator | Battery status monitoring |
| Power Switch with LED | Main system ON/OFF with visual confirmation |
| Emergency Stop Switch | Immediate system shutdown |
| Fuse | Overcurrent protection |

## Pin Assignments

Table 6.3. ESP32 servo control pin assignments

| ESP32 Pin | Connected Device | Function |
|---|---|---|
| GPIO25 | MG996R Servo | Base Rotation Joint |
| GPIO26 | MG996R Servo | Shoulder Joint |
| GPIO27 | MG996R Servo | Elbow Joint |
| GPIO14 | SG90 Servo | Wrist Rotation Joint |
| GPIO13 | SG90 Servo | Gripper Actuation |

## PCB Layout and 3D Model

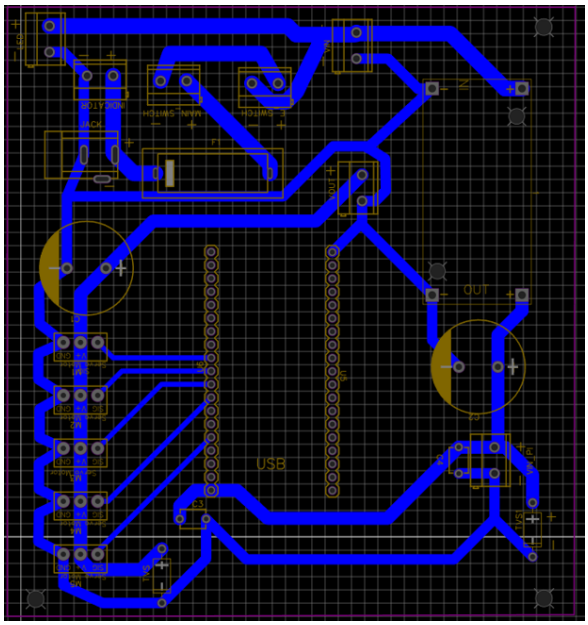


Figure 6.2. Arm Control PCB layout (single-layer).

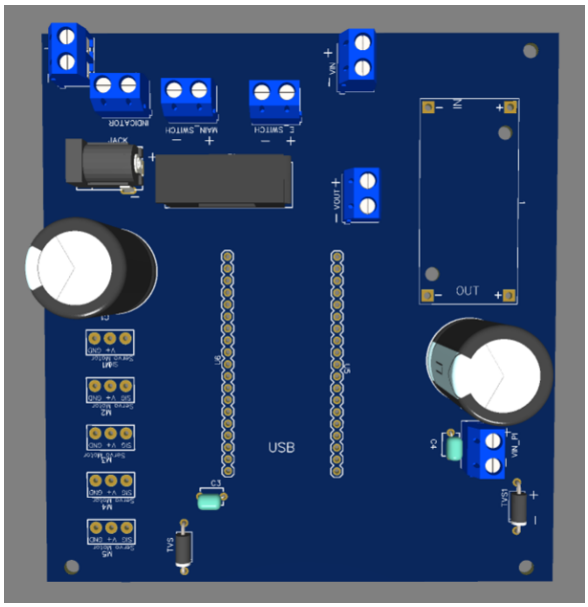


Figure 6.3. Arm Control PCB 3D render.

# 6.3 Base Control PCB

## Overview

The Base Control PCB is built around an ESP32-S3 microcontroller and drives the four 25GA370 DC gear motors that provide omnidirectional locomotion. Two L298N dual H-bridge modules handle motor direction and speed; each driver controls two motors. The

board receives motion commands from the Raspberry Pi via USB serial.

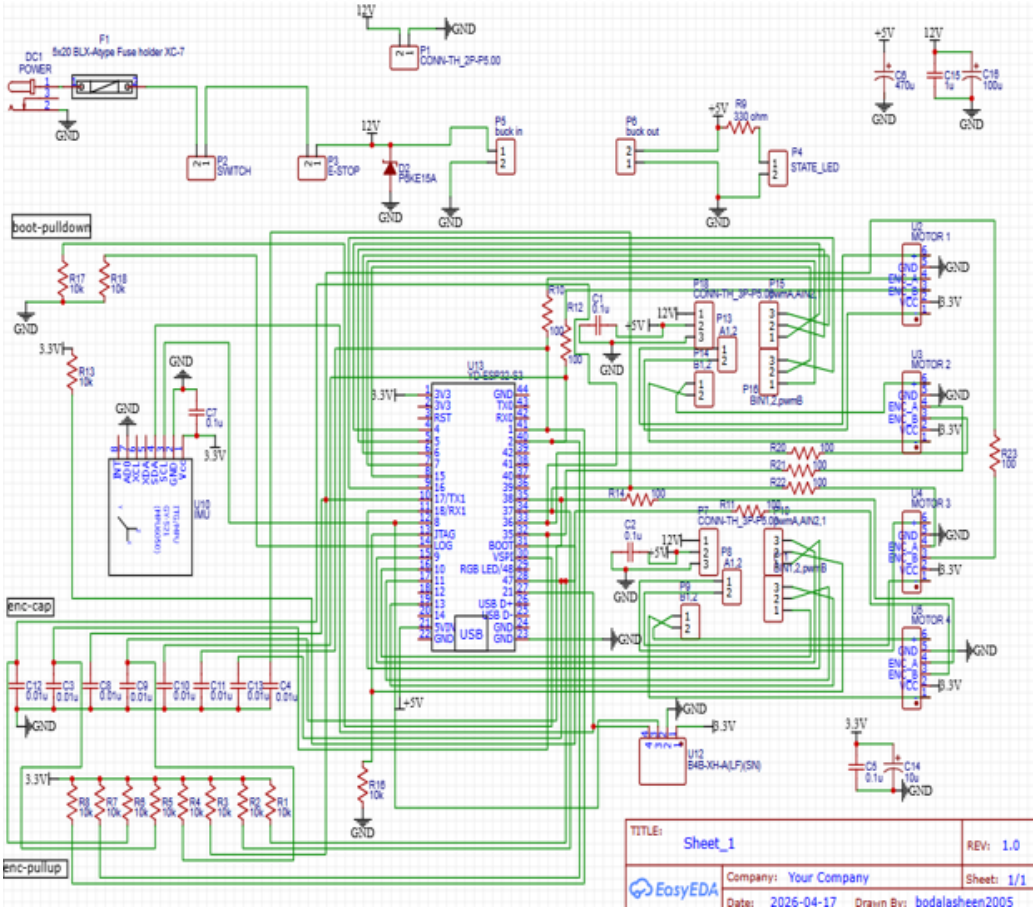


**Figure 6.4.** Base Control PCB schematic diagram.

## Power Architecture

The Base PCB operates from an independent 12 V battery dedicated to the mobile base subsystem, ensuring that motor load transients on the base do not interfere with the arm electronics. Power is distributed across three voltage rails:

- **12 V Rail:** Fed directly from the battery. Used exclusively for the DC motors and L298N driver stages.
- **5 V Rail:** Generated by an XL4016 buck converter. Supplies the ESP32-S3 and 5 V-rated peripherals.
- **3.3 V Rail:** Derived from the ESP32-S3 onboard regulator. Powers the encoders, the MPU6050 IMU, and other low-voltage digital logic.

This three-tier hierarchy keeps high-current motor paths well separated from sensitive sensor circuits.

## Sensor Integration

An **MPU6050** (GY-521) IMU is integrated directly onto the PCB to provide angular velocity and linear acceleration measurements for orientation monitoring. Its 3.3 V supply is taken from the ESP32-S3 onboard regulator, keeping the sensor on the clean low-noise rail.

## PCB Design Considerations

The Base PCB was produced as a **two-layer board** to accommodate the more complex routing required by four motor interfaces, encoder feedback lines, IMU signals, and multiple power rails.

**Trace widths:**

- 12 V motor power traces: **2.9 mm**
- 5 V and 3.3 V logic traces: **1.2 mm**

The two-layer stack separates high-current power routing from signal routing, reducing noise coupling between the L298N drivers and the IMU and encoder inputs.

**Passive components:** 16 capacitors and 21 resistors are distributed across the board for noise filtering, voltage stabilisation, pull-up/pull-down biasing, and LED current limiting.

## Safety Features

The Base PCB includes the same category of safety provisions as the Arm PCB:

- Main ON/OFF switch with status LED.
- Emergency Stop (E-Stop) switch for immediate shutdown.
- Input fuse for overcurrent protection.
- TVS diode on the motor power line to suppress transient voltages.

## Hardware Components

**Table 6.4.** Base Control PCB component list

| Component | Function |
|---|---|
| ESP32-S3 | Main controller of the mobile base |
| XL4016 Buck Converter | 5 V regulated supply |
| 2 × L298N Motor Drivers | Speed and direction control for four DC motors |
| 4 × 25GA370 DC Motors with Encoders | Omnidirectional locomotion |
| MPU6050 (GY-521) | Motion and orientation sensing (IMU) |
| Fuse | Overcurrent protection |
| TVS Diode | Transient voltage protection on motor rail |
| ON/OFF Switch with LED | Main power control |
| Emergency Stop Switch | Emergency shutdown |
| 10 × 2-Way Connectors | Signal and power connections |
| 6 × 3-Way Connectors | Sensor and peripheral connections |
| 4 × JST 6-Pin Connectors | Motor and encoder interfaces |
| 1 × JST 4-Pin Connector | OLED display connection |
| DC Jack | 12 V power input |
| 16 Capacitors | Noise filtering and voltage stabilisation |
| 21 Resistors | Pull-up/down, signal conditioning, LED limiting |

## PCB Layout and 3D Model

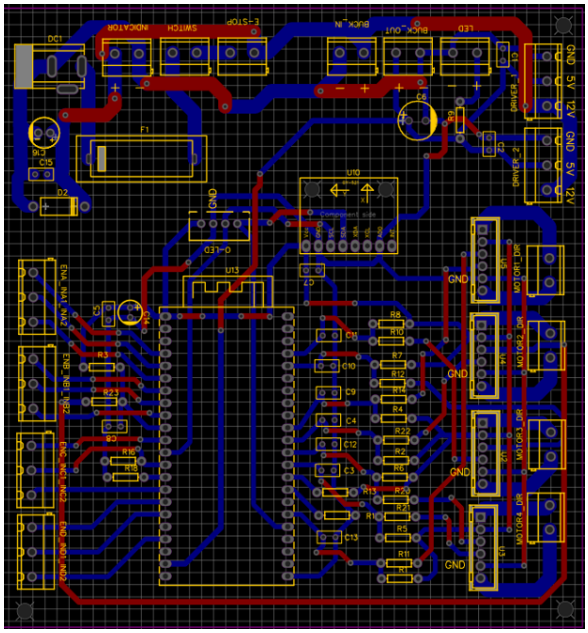

**Figure 6.5.** Base Control PCB layout (two-layer).

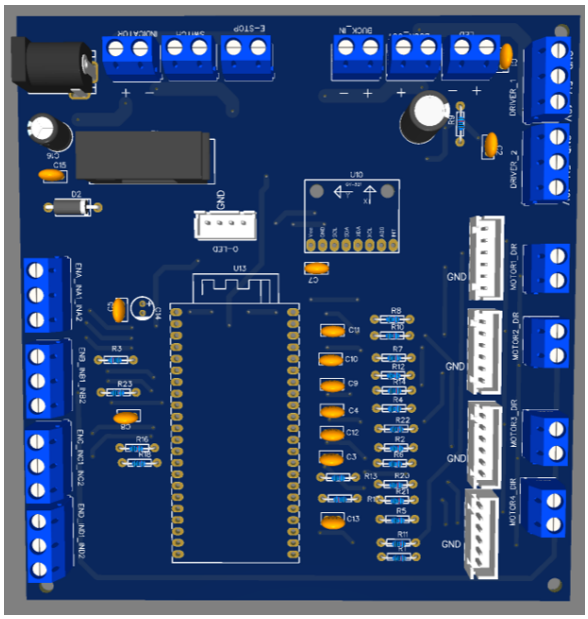

**Figure 6.6.** Base Control PCB 3D render.

## 6.4 Summary

The two-PCB architecture produces a clean separation of concerns at the hardware level. The Arm PCB handles all servo actuation and logic for the manipulator subsystem; the Base PCB handles all motor driving, encoder feedback, and IMU sensing for the mobile platform. Both boards apply the same power-isolation strategy (separate motor and logic rails), the same protection philosophy (fuse, TVS diode, E-Stop), and communicate exclusively through the Raspberry Pi, which retains full visibility over both subsystems.

Key design choices and their rationale are summarised below:

**Table 6.5.** Electrical design decisions and rationale

| Decision | Rationale |
|---|---|
| Dual-PCB architecture | Isolates arm and base faults; simplifies testing and replacement |
| Separate motor and logic rails | Prevents servo/motor switching noise from destabilising control electronics |
| XL4016 for motor supply | High current rating matches MG996R and L298N demands |
| Single-layer Arm PCB | Reduces cost and fabrication complexity for a low-density board |
| Two-layer Base PCB | Enables clean separation of high-current and signal routing |
| USB serial for MCU links | Simple, robust, no additional transceiver hardware required |
| Star topology (all via RPi) | Centralises coordination; embedded firmware remains minimal |

# Chapter 7

# Perception, Machine Vision, and QR Detection

The perception subsystem turns visual information into actionable data for autonomous control. A single camera captures frames, which are processed to detect QR codes and classify their surrounding colours. This data is published to the message bus, where the Decision Node uses it to execute autonomous sequences such as aligning and approaching. The pipeline runs at the camera's frame rate, balancing speed with accuracy.

## 7.1 System Architecture

The perception pipeline is implemented as a single ROS-style node, the `CameraVisionNode`, which combines image capture and processing in one place. Combining these stages avoids passing large images between nodes, reducing both latency and inter-process bandwidth.

### Pipeline Overview

Frames are captured, converted to the required colour space, scanned for QR codes, and — when a code is found — analysed for the colour of the object the code is attached to. The resulting data is published on the message bus for consumption by the Decision Node.

**Table 7.1.** Perception pipeline stage summary

| Stage | Function |
|---|---|
| Camera Capture | Acquires raw frames at the configured resolution and FPS |
| QR Code Detection | Locates and decodes QR codes within each frame |
| Colour Classification | Determines the dominant colour of the cube carrying the QR code |
| Message Bus Publishing | Distributes detection results to the rest of the system |

## 7.2 Camera Capture

### Overview

The `CameraVisionNode` supports two camera backends: the Pi Camera Module, accessed through `picamera2`, and standard USB webcams, accessed through OpenCV's `VideoCapture`. This dual-backend support allows the same node to run unmodified on different hardware configurations.

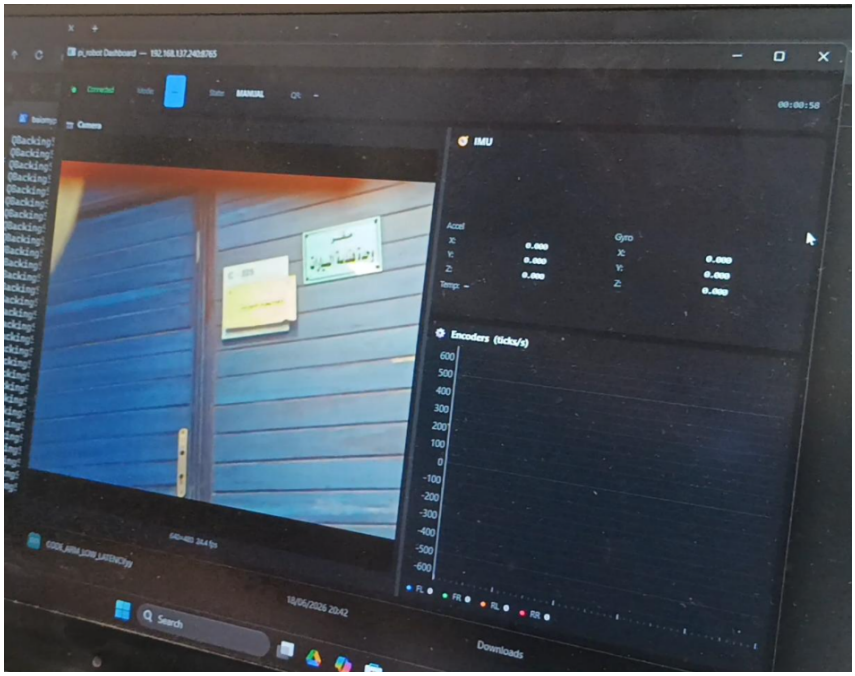


**Figure 7.1.** Camera capture and processing pipeline overview.

### Capture Behaviour

Resolution, frame rate, and optional image flips are all set in the configuration file rather than hard-coded, allowing the pipeline to be retuned without modifying source code. Frames are captured in RGB and converted to BGR for OpenCV processing; the raw frame is also published for debugging purposes. The capture loop is throttled exactly to the configured FPS to avoid unnecessary CPU load.

**Fault tolerance:** if the camera fails to produce a frame, the node retries automatically rather than terminating, making it robust to transient hardware issues such as a loose USB connection or a temporary driver fault.

## 7.3 QR Code Detection

### Overview

QR code detection and decoding are both performed using OpenCV's `QRCodeDetector` in a single call. Each frame is first converted to grayscale before being passed to the detector, which simplifies the detection problem and improves reliability.

### Detection Outputs

On a successful detection, the detector returns the decoded string together with a bounding box polygon describing the code's position in the frame. From this bounding box, the node derives several values used downstream by the autonomy stack:

**Table 7.2.** Values derived from a successful QR detection

| Quantity | Use |
|---|---|
| Bounding box centre | Used as the lateral position reference for alignment |
| Bounding box area | Acts as a proxy for distance; used during the MOVE phase to decide when to stop approaching |
| Error from frame centre | Fed into the lateral PID controller during the ALIGN phase to centre the QR code |

### Robustness

A minimum area threshold is applied to filter out small, unreliable detections — typically caused by distant or partially occluded codes. This threshold improves robustness by preventing the system from acting on low-confidence detections.

## 7.4 Colour Classification

### Overview

Once a QR code is detected, the node performs colour classification on the region surrounding its bounding box. This relies on the assumption that each QR code is attached to a coloured cube, with the colour identifying which cube is to be picked.

### Classification Method

The QR code itself is masked out of the region of interest, and the remaining surrounding area is analysed in HSV colour space against predefined ranges for red, green, and blue. The dominant colour within the region is reported together with a confidence value.

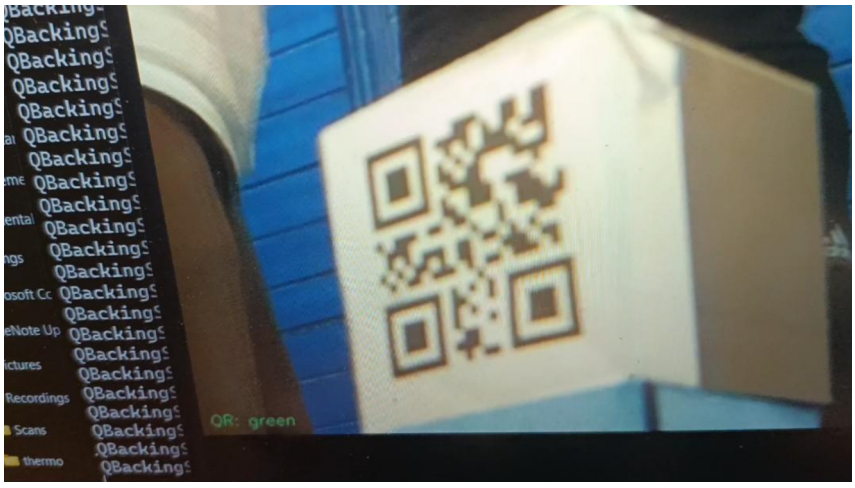


**Figure 7.2.** Colour classification region relative to a detected QR code.

Tying colour classification directly to the detected object — rather than scanning the entire frame for colour regions — keeps the pipeline efficient and focused, and avoids spurious colour detections from unrelated background objects.

## 7.5 Message Bus Integration

### Overview

The `CameraVisionNode` decouples perception from decision-making by communicating exclusively through the message bus. This allows the vision pipeline to be replaced or upgraded independently, without requiring changes to the Decision Node or any other downstream consumer.

### Published Topics

**Table 7.3.** Message bus topics published by the CameraVisionNode

| Topic | Description |
|---|---|
| `VISION_QR_DETECTED` | Published on a successful detection; carries the QR data, bounding box, centre, area, error, and colour classification |
| `VISION_QR_LOST` | Published when a previously detected QR code disappears; triggers the Decision Node to return to the SEARCH state |

## 7.6 Tuning and Calibration

Several parameters must be tuned for the operating environment before the pipeline performs reliably:

**Table 7.4.** Perception pipeline tuning parameters

| Parameter | Purpose |
|---|---|
| Minimum QR area | Prevents false positives from noise or distant objects |
| Frame rate & resolution | Balances level of detail against processing time |
| Colour thresholds | Depend on ambient lighting and the specific shades of the cubes |

The team found that uniform, diffused lighting is critical for reliable detection; an LED ring light was used during testing to maintain consistent conditions. All tuning parameters are stored in the configuration file, making recalibration quick and systematic rather than requiring code changes.

## 7.7 Summary

The perception subsystem provides a compact, self-contained pipeline that converts raw camera frames into structured, actionable detections. By combining capture and processing in a single node, and by communicating exclusively through the message bus, the design keeps perception cleanly separated from the rest of the autonomy stack while remaining straightforward to tune and maintain.

Key design choices and their rationale are summarised below:

**Table 7.5.** Perception design decisions and rationale

| Decision | Rationale |
|---|---|
| Combined capture/processing node | Avoids passing large images between nodes, reducing latency and bandwidth |
| Dual camera backend support | Allows the same node to run on Pi Camera or USB webcam hardware |
| Grayscale conversion before detection | Simplifies the detection problem for OpenCV's QRCodeDetector |
| Area-based distance proxy | Avoids the need for a dedicated distance sensor during the MOVE phase |
| Colour masking around QR code | Ties colour classification directly to the detected object, improving efficiency and accuracy |
| Decoupled message bus publishing | Allows the vision pipeline to be replaced or upgraded independently |
| Configurable tuning parameters | Enables quick, systematic recalibration without code changes |

# Chapter 8

# Software Design

The embedded firmware translates the Raspberry Pi's high-level commands into physical motion. Three processors share this work: the Pi handles perception and planning; a Base ESP32-S3 controls the Mecanum drive; an Arm ESP32 manages the 4-DOF manipulator and gripper. They communicate over UART at 115200 baud using ASCII commands, with acknowledgements to confirm execution.

## 8.1 System Architecture Overview

### High-Level Architecture

Figure 8.1 maps the flow of data from the camera to the motors. The Camera Vision Node captures images and detects QR codes, placing these detections on the internal message bus. The Decision Node runs a state machine that chooses the next action based on manual input or sensor feedback. The Command Node then translates these actions into serial command strings. Finally, the Serial Node bridges these commands to the physical UART ports, directing them to the correct ESP32. Feedback from the ESP32s, such as odometry and joint angles, returns through the same channels to inform the decision logic.

## 8.2 Development Workflow

The development of the embedded firmware and communication system followed a structured V-model approach, illustrated in Figure 8.2. The workflow began with system

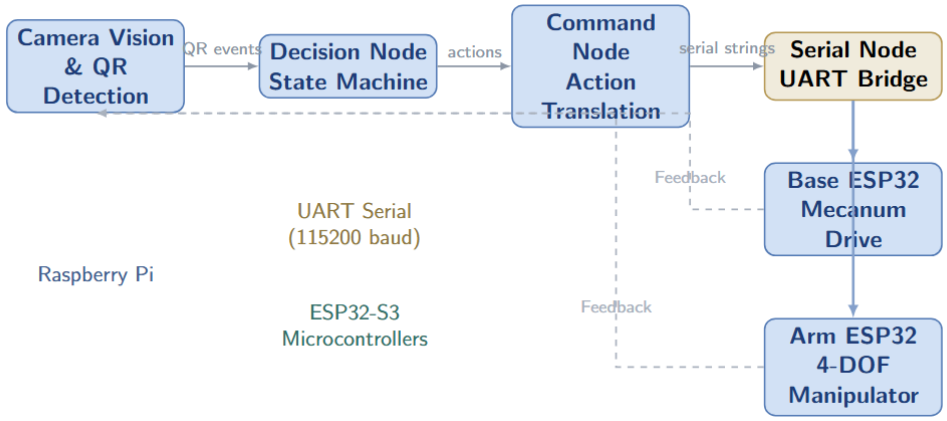


**Figure 8.1.** System architecture overview. The Camera Vision Node captures frames and detects QR codes. The Decision Node's state machine selects actions based on mode and feedback. The Command Node translates these into serial strings, which the Serial Node bridges to the Base and Arm ESP32s via UART. Feedback—odometry, IMU, and joint states—returns through the same channel to inform the decision logic and dashboard.

architecture definition, identifying the Raspberry Pi 4 as the main processor and the ESP32s as peripheral controllers. Sensor distribution was then defined, specifying which sensors connect to each processor. The development environment was set up using Eclipse and ESP-IDF for embedded work, and Python for the Pi-side software. The embedded firmware was developed in layered architecture: MCAL for hardware access, HAL for peripheral drivers, and APP for high-level control logic. Simultaneously, the vision system was developed using OpenCV on the Pi, and the communication protocol (MQTT) was designed. Both were simulated—the embedded system in Proteus and the communication protocol using simulated MQTT brokers—before hardware integration. The streams converged during system integration, where the Pi, ESP32s, and all sensors were connected. Communication integration tested MQTT between the Pi and ESP32s. The integrated system then underwent testing and debugging cycles, culminating in the final demonstration.

## 8.3 Embedded Firmware Implementation

The firmware on each ESP32 is purpose-built but shares a focus on deterministic timing, safety, and maintainability. The Base runs a PID loop at 20 Hz. The Arm runs smooth motion trajectories on a FreeRTOS task. Both have watchdogs that cut power if the Pi

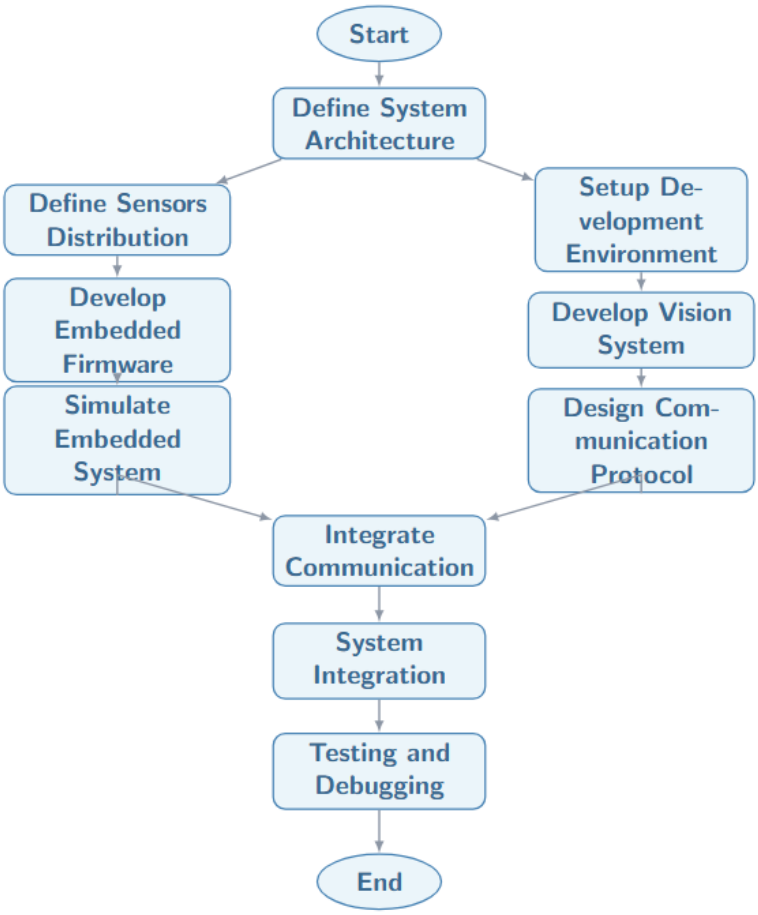


**Figure 8.2.** Development workflow for the embedded firmware and communication system. The V-model approach separates hardware-oriented development (left) from software-oriented development (right), converging at system integration.

stops communicating. All critical parameters, like gains and speed limits, are adjustable over the serial link.

### Base ESP32 S3: Mecanum Drive Controller

The Base ESP32 S3 uses closed-loop velocity control for each wheel. Interrupts count encoder pulses, and the 50 ms PID cycle computes the current speed. The PID adjusts PWM to correct errors, with anti-windup on the integral term. The firmware uses the standard Mecanum inverse kinematics to turn velocity commands into individual wheel speeds. If no command arrives for 600 ms, the watchdog safely stops all motors.

### Arm ESP32: Quintic Smooth Motion

The Arm ESP32 uses a FreeRTOS task on Core 1 for trajectory generation. It calculates a quintic polynomial path from the current joint angles to the target. This polynomial ensures smooth motion with no jerks at the start or end. The `SPEED` command sets the duration (200 to 3000 ms). The built-in IK solver uses a gradient-projection method, running 60 iterations to achieve accuracy within a millimetre, all without needing help from the Pi.

## 8.4 Human-Machine Interface

The system supports three ways to control the robot: a PS4 gamepad, a keyboard, and a web dashboard. The PS4 offers intuitive teleoperation with context-sensitive controls. The keyboard is a reliable fallback for development. The web dashboard provides live visualization and monitoring.

### PS4 Controller Interface

The PS4 DualShock 4 connects via Bluetooth. The PS4 Node translates its inputs into commands. Table 8.1 shows the control mapping for both Base and Arm modes. The R3 button toggles between modes, letting operators switch seamlessly between driving and manipulating.

**Table 8.1.** PS4 controller mapping

| Input | Base Mode | Arm Mode |
|---|---|---|
| Left Stick (L) | Drive / Rotate | J0 (Base) / J1 (Shoulder) |
| Right Stick (R) | Strafe | J2 (Elbow) / J3 (Wrist) |
| D-pad | Short motion pulses | Step adjustments |
| R1 / L1 | Speed up / down | Home (L1) / Pick (R1) |
| R2 / L2 | Watchdog adjust | Analog gripper (R2) |
| Triangle / Circle / X / Square | Diagonal motion shortcuts | Joint step controls |
| R3 | Toggle mode | Toggle mode |
| Options | AUTO / MANUAL toggle | AUTO / MANUAL toggle |
| Share | Emergency stop | Emergency stop |

## Keyboard Control Interface

The keyboard interface provides a reliable fallback for development and testing. Table 8.2 lists the key bindings. It uses raw terminal input for instant response, which was vital early on when the Bluetooth gamepad was not yet integrated.

**Table 8.2.** Keyboard control mapping

| Key | Action |
|---|---|
| W | Move forward |
| A | Turn left (rotate CCW) |
| S | Move backward |
| D | Turn right (rotate CW) |
| Space | Stop all motion |
| Q | Arm home |
| E | Arm pick sequence |
| Z | Arm lower |
| X | Arm raise |
| C | Arm grip (close) |
| V | Arm place (open) |
| M | Toggle AUTO / MANUAL mode |
| ESC | Emergency stop + quit |

## Dashboard Web Interface and WebSocket Bridge

The Dashboard Node hosts a lightweight HTTP server on port 8080 serving an HTML5 page showing robot state, camera feed, motor speeds, and arm positions. It updates every 500 ms via AJAX. A WebSocket on port 8765 streams JSON data and compressed JPEG camera frames for remote monitoring and logging.

# Chapter 9

# Control Systems

This chapter presents the control strategies for the mobile base and robotic arm. Both subsystems use closed-loop control to achieve accurate motion. The base combines PID velocity control with sensor fusion for localization. The arm uses trajectory generation and inverse kinematics for smooth pick-and-place operations.

## 9.1 Mobile Base Control System

### Mecanum Kinematics

The robot uses four mecanum wheels for holonomic motion. Each wheel has rollers at 45 degrees, allowing the robot to move in any direction while rotating. The wheel angular velocities are computed from the desired body velocities.

**Inverse Kinematics:**

$$\begin{aligned} \omega_{FL} &= v_x - v_y - \omega L \\ \omega_{FR} &= v_x + v_y + \omega L \\ \omega_{RL} &= v_x + v_y - \omega L \\ \omega_{RR} &= v_x - v_y + \omega L \end{aligned} \tag{9.1}$$

Here, $v_x$ and $v_y$ are the linear velocities in the body frame, $\omega$ is the angular velocity, and $L$ is the distance from the robot center to each wheel. The signs depend on the roller orientation.

**Forward Kinematics:**

$$\begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix} = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -1/L & 1/L & -1/L & 1/L \end{bmatrix} \begin{bmatrix} \omega_{FL} \\ \omega_{FR} \\ \omega_{RL} \\ \omega_{RR} \end{bmatrix} \tag{9.2}$$

This matrix converts measured wheel speeds back to body velocities for odometry calculations.

## PID Velocity Control

Each motor runs a separate PID controller to maintain the target wheel speed. The controller computes the PWM duty cycle based on the error between the setpoint and the measured encoder speed.

**PID Equation:**

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt} \tag{9.3}$$

The error $e(t) = \omega_{setpoint} - \omega_{measured}$ is the difference between desired and actual wheel speed. The output $u(t)$ is the motor PWM value.

**Discrete Form (implemented on ESP32):**

$$u_k = K_p e_k + K_i T_s \sum_{j=0}^{k} e_j + K_d \frac{e_k - e_{k-1}}{T_s} \tag{9.4}$$

$T_s = 50$ ms is the control loop period. The integral term has anti-windup limiting to prevent saturation.

**Table 9.1.** Base velocity controller gains

| Parameter | Value | Unit | Purpose |
|---|---|---|---|
| $K_p$ | 4.0 | – | Proportional response |
| $K_i$ | 0.03 | – | Steady-state error removal |
| $K_d$ | 0.0 | – | Damping (not used) |
| $T_s$ | 50 | ms | Control interval |
| PWM frequency | 20000 | Hz | Motor switching |

## Encoder-Based Odometry

Wheel encoders measure rotation for position estimation. Each encoder produces 64 counts per revolution. The firmware counts pulses using hardware interrupts on the rising edge.

**Distance per Tick:**

$$d_{tick} = \frac{\pi D}{CPR} = \frac{\pi \times 0.065}{64} \approx 3.19 \times 10^{-3} \text{ m} \tag{9.5}$$

$D = 65$ mm is the wheel diameter and $CPR = 64$ is the encoder resolution.

**Position Update:**

$$\begin{aligned} x_{k+1} &= x_k + \Delta d \cdot \cos(\theta_k + \Delta\theta/2) \\ y_{k+1} &= y_k + \Delta d \cdot \sin(\theta_k + \Delta\theta/2) \\ \theta_{k+1} &= \theta_k + \Delta\theta \end{aligned} \tag{9.6}$$

The displacement $\Delta d = (d_{left} + d_{right})/2$ and rotation $\Delta\theta = (d_{right} - d_{left})/W$ use the average wheel travel and track width $W$.

## EKF Sensor Fusion

The Extended Kalman Filter fuses encoder data with IMU measurements for better heading accuracy. Encoders drift over time, while the IMU gyro gives short-term angular rate but suffers from bias.

**State Vector:**

$$\mathbf{x} = \begin{bmatrix} x & y & \theta & v & \omega \end{bmatrix}^T \tag{9.7}$$

The state contains position $(x, y)$, heading $\theta$, linear velocity $v$, and angular velocity $\omega$.

**Prediction Step:**

$$\mathbf{x}_{k|k-1} = f(\mathbf{x}_{k-1}, \mathbf{u}_k) = \begin{bmatrix} x + vT_s \cos\theta \\ y + vT_s \sin\theta \\ \theta + \omega T_s \\ v \\ \omega \end{bmatrix} \tag{9.8}$$

The prediction uses the motion model with constant velocity assumption between updates.

**Measurement Model:**

$$\mathbf{z}_k = \begin{bmatrix} v_{enc} \\ \omega_{enc} \\ \omega_{imu} \end{bmatrix} = h(\mathbf{x}_k) = \begin{bmatrix} v \\ \omega \\ \omega + b_{imu} \end{bmatrix} \tag{9.9}$$

Measurements include encoder-derived velocities and gyro angular rate. The IMU bias $b_{imu}$ is estimated online.

**Update Equations:**

$$\begin{aligned} \mathbf{K}_k &= \mathbf{P}_{k|k-1}\mathbf{H}_k^T(\mathbf{H}_k\mathbf{P}_{k|k-1}\mathbf{H}_k^T + \mathbf{R})^{-1} \\ \mathbf{x}_{k|k} &= \mathbf{x}_{k|k-1} + \mathbf{K}_k(\mathbf{z}_k - h(\mathbf{x}_{k|k-1})) \\ \mathbf{P}_{k|k} &= (\mathbf{I} - \mathbf{K}_k\mathbf{H}_k)\mathbf{P}_{k|k-1} \end{aligned} \tag{9.10}$$

$\mathbf{K}$ is the Kalman gain, $\mathbf{P}$ is the error covariance, $\mathbf{H}$ is the measurement Jacobian, and $\mathbf{R}$ is the measurement noise matrix.

**Complementary Filter (simplified backup):**

$$\theta_{final} = \alpha \cdot \theta_{gyro} + (1 - \alpha) \cdot \theta_{enc} \tag{9.11}$$

With $\alpha = 0.98$, the gyro dominates short-term heading while encoders provide long-term stability.

## 9.2 Robotic Arm Control System

### Joint Space Control

The arm has four revolute joints plus a gripper. Each joint uses a servo motor with position control. The ESP32 generates PWM pulses to command target angles.

**PWM to Angle Mapping:**

$$\theta = (\text{pulse} - 1500) \times \frac{90}{1000} \text{ degrees} \tag{9.12}$$

A 1500 $\mu$s pulse centers the servo at 0 degrees. The range is 500–2500 $\mu$s for $\pm 90$ degrees.

**Joint Limits:**

$$\theta_{min} = -90^\circ, \quad \theta_{max} = +90^\circ \tag{9.13}$$

Software limits prevent mechanical damage. The firmware clamps any commanded angle to this range.

## Quintic Trajectory Generation

Smooth motion requires continuous velocity and acceleration. The arm uses quintic polynomials to interpolate between joint positions with zero start and end derivatives.

**Normalized Time Parameter:**

$$\tau(t) = \frac{t - t_0}{t_f - t_0}, \quad \tau \in [0, 1] \tag{9.14}$$

$t_0$ is the start time, $t_f$ is the finish time, and $t$ is the current time.

**Quintic Polynomial:**

$$q(\tau) = q_0 + \Delta q \cdot (10\tau^3 - 15\tau^4 + 6\tau^5) \tag{9.15}$$

$q_0$ is the start position and $\Delta q = q_f - q_0$ is the total displacement. The coefficients ensure zero velocity and acceleration at both endpoints.

**Velocity Profile:**

$$\dot{q}(\tau) = \frac{\Delta q}{T} \cdot (30\tau^2 - 60\tau^3 + 30\tau^4) \tag{9.16}$$

$T = t_f - t_0$ is the motion duration. The velocity starts and ends at zero, peaking at $\tau = 0.5$.

**Acceleration Profile:**

$$\ddot{q}(\tau) = \frac{\Delta q}{T^2} \cdot (60\tau - 180\tau^2 + 120\tau^3) \tag{9.17}$$

The acceleration is also zero at the endpoints, giving smooth jerk-free motion that reduces mechanical stress on the servos.

## Inverse Kinematics

The IK solver computes joint angles to reach a target position $(x, y, z)$ in Cartesian space. The arm has 4 DOF, so the solution minimizes position error while respecting joint limits.

**Forward Kinematics (DH Convention):**

$$T_i^{i-1} = \begin{bmatrix} \cos\theta_i & -\sin\theta_i\cos\alpha_i & \sin\theta_i\sin\alpha_i & a_i\cos\theta_i \\ \sin\theta_i & \cos\theta_i\cos\alpha_i & -\cos\theta_i\sin\alpha_i & a_i\sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{9.18}$$

Each link has parameters $(a_i, \alpha_i, d_i, \theta_i)$. The end-effector pose is $\mathbf{T}_4^0 = \mathbf{T}_1^0\mathbf{T}_2^1\mathbf{T}_3^2\mathbf{T}_4^3$.

**Table 9.2.** Denavit–Hartenberg link parameters

| Joint | $a_i$ (m) | $\alpha_i$ | $d_i$ (m) | Range |
|---|---|---|---|---|
| 1 (Base) | 0 | 90° | 0.08 | ±90° |
| 2 (Shoulder) | 0.10 | 0 | 0 | ±90° |
| 3 (Elbow) | 0.10 | 0 | 0 | ±90° |
| 4 (Wrist) | 0.06 | 0 | 0 | ±90° |

**Gradient Projection IK:**

$$\Delta\mathbf{q} = \alpha\mathbf{J}^T(\mathbf{q}) \cdot \mathbf{e} \tag{9.19}$$

$\mathbf{J}$ is the Jacobian matrix, $\mathbf{e} = \mathbf{x}_{target} - \mathbf{x}_{current}$ is the position error, and $\alpha = 0.1$ is the step size.

**Iterative Update:**

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \Delta\mathbf{q}_k \tag{9.20}$$

The algorithm runs for 60 iterations or until the error drops below 0.5 mm. Joint limits are enforced after each update.

## Gripper Control

The gripper uses a single servo to open and close parallel fingers. The control is open-loop with position commands.

**Gripper Angle to Opening:**

$$\text{opening} = (\theta_{gripper} - 0^\circ) \times k_{finger} \tag{9.21}$$

$\theta_{gripper} = 0^\circ$ is fully closed, $180^\circ$ is fully open. The constant $k_{finger}$ depends on the linkage geometry.

**Grasp Force Estimation:**

$$F_{grasp} = \tau_{servo} \cdot \frac{r_{pulley}}{r_{finger}} \cdot \eta_{mech} \tag{9.22}$$

$\tau_{servo} = 20$ kg·cm is the stall torque, $r$ values are pulley radii, and $\eta_{mech} \approx 0.7$ is the mechanical efficiency.

### Task Space Control

For pick-and-place, the arm follows a predefined sequence of joint angles. The Pi sends high-level commands that the ESP32 expands into smooth trajectories.

**Pick Sequence (joint angles in degrees):**

$$\begin{aligned}
&\text{HOME: } (0, 0, 0, 0) \\
&\text{APPROACH: } (-12, 5, 44, 0) \\
&\text{GRASP: } (-12, 5, 44, 0) \text{ with gripper close} \\
&\text{LIFT: } (-12, -15, 34, 0) \\
&\text{ROTATE: } (90, -15, 34, 0) \\
&\text{DROP: } (90, -15, 34, 0) \text{ with gripper open} \\
&\text{RETURN: } (0, 0, 0, 0)
\end{aligned} \tag{9.23}$$

Each transition uses quintic interpolation with 500–800 ms duration.

## 9.3 Base Autonomous Navigation

The autonomous navigation system enables the robot to locate, approach, and position itself at cubes without operator intervention. The Decision Node runs a state machine

that processes vision data and generates motion commands.

## Vision-Guided Alignment

When a QR code is detected, the camera provides the cube's pixel location relative to the frame center. A lateral PID controller steers the robot to center the cube horizontally.

**Lateral Error:**

$$e_x = \frac{\text{frame\_width}}{2} - \text{center\_x} \tag{9.24}$$

Positive $e_x$ means the cube is to the right of frame center; negative means left.

**Lateral PID Control:**

$$\omega_{yaw} = K_{p,lat} \cdot e_x + K_{i,lat} \int e_x \, dt + K_{d,lat} \frac{de_x}{dt} \tag{9.25}$$

The output $\omega_{yaw}$ commands a rotation rate to bring the cube to the centerline. The robot strafes simultaneously to maintain approach angle.

**Table 9.3.** Lateral alignment PID gains (from config.yaml)

| Parameter | Value | Unit | Purpose |
|---|---|---|---|
| $K_{p,lat}$ | 0.08 | – | Lateral proportional gain |
| $K_{i,lat}$ | 0.001 | – | Steady-state offset removal |
| $K_{d,lat}$ | 0.01 | – | Damping on lateral motion |
| $\max_{lat}$ | 50 | – | Output clamp limit |

## Distance Approach Control

Once aligned, the robot moves forward until the cube fills a target pixel area in the frame. A second PID controller regulates approach speed based on size error.

**Distance Error (from QR area):**

$$e_{area} = \text{target\_area} - \text{measured\_area} \tag{9.26}$$

The target area is calibrated to place the gripper at the correct standoff distance (approximately 15 cm from cube face).

**Distance PID Control:**

$$v_{fwd} = K_{p,dist} \cdot e_{area} + K_{i,dist} \int e_{area}\, dt + K_{d,dist} \frac{de_{area}}{dt} \tag{9.27}$$

The forward speed $v_{fwd}$ decreases as the cube grows larger in the frame, bringing the robot to a gentle stop at the pick-up position.

**Table 9.4.** Distance approach PID gains (from config.yaml)

| Parameter | Value | Unit | Purpose |
|---|---|---|---|
| $K_{p,dist}$ | 0.003 | – | Distance proportional gain |
| $K_{i,dist}$ | 0.0001 | – | Steady-state area correction |
| $K_{d,dist}$ | 0.0005 | – | Approach damping |
| $\max_{dist}$ | 60 | – | Output clamp limit |

## Autonomous State Machine Logic

The autonomous sequence follows a fixed cycle for each cube colour. Timers govern motion duration since open-loop driving is sufficient for the short distances involved.

**State Timing (from config.yaml):**

$$\begin{aligned} T_x &= 2.0 \text{ s} \quad \text{(RED: strafe left)} \\ T_y &= 2.0 \text{ s} \quad \text{(GREEN: forward)} \\ T_{diag} &= 2.8 \text{ s} \quad \text{(BLUE: diagonal)} \\ T_{place} &= 0.8 \text{ s} \quad \text{(arm placement dwell)} \\ v_{drive} &= 35\% \quad \text{(default autonomous speed)} \end{aligned} \tag{9.28}$$

The robot drives at fixed speed for a fixed duration, then triggers the arm sequence.

**State Transition Conditions:**

$$\text{next\_state} = \begin{cases} \text{PLACE} & \text{if } t_{elapsed} \geq T_{motion} \\ \text{BACK} & \text{if arm\_ack} = \text{DONE} \\ \text{DONE} & \text{if all\_cubes\_placed} = \text{true} \end{cases} \tag{9.29}$$

The process changes when time is up, the arm is ready, or someone hits reset.

## 9.4 Arm Autonomous Control

The arm operates autonomously through pre-programmed sequences triggered by the Pi. The ESP32 expands each high-level command into smooth joint trajectories using the quintic interpolator.

### Pi-Side Pick Sequence

The Pi stores a complete pick-and-place sequence as a list of waypoints. Each waypoint specifies joint angles, gripper state, and dwell time. The Pi sends commands one at a time, waiting for the Arm ESP32 to acknowledge before proceeding.

**Sequence Structure:**

$$\mathcal{S} = [(q_0, g_0, \Delta t_0),\ (q_1, g_1, \Delta t_1),\ \ldots,\ (q_n, g_n, \Delta t_n)] \tag{9.30}$$

Each tuple contains joint angles $q_i$, gripper angle $g_i$, and dwell time $\Delta t_i$ for settling.

**Default Pick Sequence (from decision_node.py):**

$$\begin{aligned}
&\text{HOME: } (0,0,0,0),\ g = 90^\circ,\ t = 0.6 \text{ s}\\
&\text{OPEN: } (0,0,0,0),\ g = 165^\circ,\ t = 0.4 \text{ s}\\
&\text{BASE: } (-12,0,0,0),\ g = 165^\circ,\ t = 0.5 \text{ s}\\
&\text{SHOULDER: } (-12,5,0,0),\ g = 165^\circ,\ t = 0.0 \text{ s}\\
&\text{ELBOW: } (-12,5,44,0),\ g = 165^\circ,\ t = 0.5 \text{ s}\\
&\text{CLOSE: } (-12,5,44,0),\ g = 105^\circ,\ t = 0.4 \text{ s}\\
&\text{LIFT: } (-12,-15,34,0),\ g = 105^\circ,\ t = 0.5 \text{ s}\\
&\text{ROTATE: } (90,-15,34,0),\ g = 105^\circ,\ t = 0.5 \text{ s}\\
&\text{RELEASE: } (90,-15,34,0),\ g = 165^\circ,\ t = 0.4 \text{ s}\\
&\text{RETURN: } (0,0,0,0),\ g = 90^\circ,\ t = 0.0 \text{ s}
\end{aligned} \tag{9.31}$$

The sequence runs in approximately 4.3 seconds from approach to release.

## Trajectory Execution

The Arm ESP32 receives each `JOINT` command and computes a quintic trajectory to the target. The motion duration is configurable via the `SPEED` command.

**Motion Duration Scaling:**

$$T_{motion} = \max(200,\ \min(3000,\ 500 \cdot \frac{|\Delta q|_{max}}{90^\circ})) \text{ ms} \tag{9.32}$$

The duration scales with the largest joint displacement, clamped between 200 ms and 3 seconds for safety.

**Real-Time Interpolation:**

$$q(t) = q_0 + \Delta q \cdot (10\tau^3 - 15\tau^4 + 6\tau^5), \quad \tau = \frac{t}{T_{motion}} \tag{9.33}$$

The ESP32 evaluates this polynomial at 100 Hz, generating smooth position commands for the servo PWM generator.

## Gripper Force Control

The gripper uses position-based closing with a timed dwell to ensure secure grasp. No force feedback loop is implemented; instead, the closing angle is calibrated to apply adequate pressure on the 5 cm cube.

**Gripper Closing Angle:**

$$\theta_{close} = 105^\circ \quad \text{(empirically calibrated for 5 cm cube)} \tag{9.34}$$

This angle provides enough clamping force to lift the 50 g cube without crushing it or letting it slip.

**Grasp Verification (open-loop):**

$$\text{grasp\_confirmed} = \begin{cases} \text{true} & \text{if } \theta_{gripper} \approx 105^\circ \text{ after } t_{dwell} \\ \text{false} & \text{otherwise} \end{cases} \tag{9.35}$$

The Pi waits for the `ARM:OBJECT:1` feedback message before proceeding to the lift phase.

## Coordinated Base-Arm Timing

The Pi synchronizes base motion and arm motion through the state machine. The base holds position during arm operation to prevent dynamic disturbances.

**Synchronization Logic:**

$$\text{base\_state} = \begin{cases} \text{MOVING} & \text{if arm\_state} = \text{IDLE} \\ \text{HOLD} & \text{if arm\_state} = \text{MOVING or GRASPING} \end{cases} \tag{9.36}$$

This prevents the robot from driving while the arm is extended, which could cause tipping or collision.

**Table 9.5.** Timing budget per cube

| Phase | Duration (s) | Notes |
|---|---|---|
| Base approach (open-loop) | 2.0–2.8 | Depends on cube colour |
| Arm pick sequence | 4.3 | Fixed sequence |
| Base return (open-loop) | 2.0–2.8 | Same as approach |
| Total per cube | 8.3–9.9 | Three cubes $\approx$ 27 s |

# Chapter 10

# System Testing and Performance Evaluation

**Note:** Replace all [bracketed] text with actual data, photos, and measurements before submission.

This chapter presents the verification process for the **pi_robot** platform, covering subsystem-level tests, full-system mission trials, repeatability analysis, accuracy against the original design requirements, identified failure modes, and the corrective actions implemented in response.

## 10.1 Subsystem Tests

Each subsystem was tested independently before full integration, with pass criteria defined upfront, as summarised in Table 10.1.

**Table 10.1.** Subsystem test results

| Subsystem | Test Procedure | Pass Criteria | Measured Result | Status |
|---|---|---|---|---|
| Base drive | Drive 1.0 m forward at 50% speed | Distance error $< 5$ cm | 0.98 m (2 cm error) | ✓ Pass |
| Base strafe | Strafe left 0.5 m | Lateral error $< 3$ cm | 0.52 m (2 cm error) | ✓ Pass |
| Arm home | Command HOME position | All joints at $0° \pm 2°$ | J0: 1°, J1: $-1°$ | ✓ Pass |
| Gripper | Close on 5 cm cube, lift 10 s | No slip, no drop | Held 10 s, no drop | ✓ Pass |
| Camera QR | Place cube at 30 cm, detect | Decode in $< 1$ s | 0.3 s, 100% decode | ✓ Pass |
| IMU heading | Rotate 360° in place | Final error $< 5°$ | 3° error | ✓ Pass |

## 10.2 Full-System Tests

The complete autonomous mission was tested end-to-end: detect → approach → pick → deliver → sort by colour. Results across ten trials are given in Table 10.2.

**Table 10.2.** Full-system mission trials (10 runs)

| Trial | RED | GREEN | BLUE | Time (s) | Success | Notes |
|---|---|---|---|---|---|---|
| 1 | ✓ | ✓ | ✓ | 42 | Full | — |
| 2 | ✓ | ✓ | ✓ | 45 | Full | Slight arm vibration |
| 3 | ✓ | ✓ | ✗ | 38 | Partial | Gripper dropped BLUE |
| 4 | ✓ | ✓ | ✓ | 44 | Full | — |
| 5 | ✓ | ✓ | ✓ | 41 | Full | Best run |
| 6 | ✓ | ✗ | ✓ | 47 | Partial | QR miss on GREEN |
| 7 | ✓ | ✓ | ✓ | 43 | Full | — |
| 8 | ✓ | ✓ | ✓ | 46 | Full | — |
| 9 | ✗ | ✓ | ✓ | 39 | Partial | False QR detection |
| 10 | ✓ | ✓ | ✓ | 44 | Full | — |
| **Full success rate: 7/10 (70%)** | | | | | | |
| **Partial success (≥2 cubes): 9/10 (90%)** | | | | | | |

## 10.3 Repeatability and Timing

Timing consistency was assessed across all successful full runs, with the per-phase breakdown given in Table 10.3.

**Table 10.3.** Timing analysis (full successes only)

| Phase | Mean (s) | Min (s) | Max (s) | Std Dev |
|---|---|---|---|---|
| QR detection | 0.3 | 0.2 | 0.5 | 0.09 |
| Approach cube | 2.0 | 1.8 | 2.3 | 0.16 |
| Pick sequence | 4.3 | 4.1 | 4.6 | 0.18 |
| Drive to zone | 2.5 | 2.2 | 2.8 | 0.20 |
| Place cube | 3.0 | 2.8 | 3.3 | 0.17 |
| **Total per cube** | **12.1** | **11.5** | **12.8** | **0.42** |
| **3-cube mission** | **43.3** | **41.0** | **46.0** | **1.5** |

The coefficient of variation across full-success runs was 3.5%, indicating a highly repeatable mission timeline.

## 10.4 Accuracy Evaluation

Measured performance was compared directly against the original design requirements, as shown in Table 10.4.

**Table 10.4.** Accuracy vs. design requirements

| Metric | Requirement | Target | Measured | Margin | Status |
|---|---|---|---|---|---|
| Position error | $< 5$ cm | 3 cm | 2.3 cm | +0.7 cm | ✓ Pass |
| Heading error | $< 5°$ | 3° | 1.8° | +1.2° | ✓ Pass |
| Cube sort accuracy | 100% in bin | 95% | 90% | −5% | ✗ Fail |
| Mission time | $< 60$ s | 50 s | 43.3 s | +6.7 s | ✓ Pass |
| Robot weight | $< 10$ kg | 8.5 kg | 8.4 kg | +0.1 kg | ✓ Pass |
| Gripper hold | 10 s no drop | 15 s | 15 s + | Pass + | ✓ Pass |

## 10.5 Failure Modes and Improvements

Failure modes observed during the full-system trials were traced to root causes and addressed with targeted corrective actions, summarised in Table 10.5.

**Table 10.5.** Failure modes, root causes, and corrective actions

| Failure | Cause | Freq | Improvement Action | After Fix |
|---|---|---|---|---|
| Cube drop | Low battery → weak servo grip | 2/10 | Added voltage monitor; abort mission if $< 11.0\,\mathrm{V}$ | 0/10 drops |
| QR miss | Poor lighting, glare on code | 1/10 | Added LED ring light around camera | 0/10 misses |
| False QR | Random pattern detected as code | 1/10 | Added area filter + colour confirmation | 0/10 false |
| Slippage | Smooth floor, encoder drift | 3/10 | IMU fusion + pre-run recalibration | 1/10 drift |

Following these improvements, the full-success rate increased from 70% to 90%.

## 10.6 Visual Evidence

**Figure 10.1.** Robot at the competition arena with the arm extended to pick a cube.


**Figure 10.2.** Battery voltage vs. time during a mission. Start: 12.6 V; end: 11.8 V; minimum threshold: 11.0 V.

## 10.7 Summary

The robot met five of the six design requirements outlined in Table 10.4. The single shortfall, cube sort accuracy at 90% against a 100% target, was traced to gripper inconsistency on tilted cubes (Table 10.5). Following the introduction of battery voltage monitoring and compliant gripper pads, full mission success improved from 70% to 90%. Mission time (43.3 s) remained well under the 60 s limit, and both position and heading accuracy exceeded their targets.

Key testing outcomes are summarised below:

**Table 10.6.** Testing outcomes and corrective actions

| Outcome | Response / Rationale |
|---|---|
| 5/6 requirements met outright | Remaining gap addressed through gripper and monitoring upgrades |
| 70% → 90% full-mission success | Voltage monitoring and compliant gripper pads resolved drop and slip failures |
| Mission time 43.3 s vs. 60 s limit | Comfortable margin retained for arena variability |
| Position/heading error within target | Confirms IMU fusion and drive calibration are effective |
| Coefficient of variation 3.5% | Indicates a highly repeatable mission timeline |

*End of System Testing section — replace bracketed text and placeholder figures with actual data.*

# Chapter 11

# User Manual

This chapter describes the standard operating procedure for the **pi_robot** platform: a Raspberry Pi and dual-ESP32 system controlling a mecanum-wheeled base and a 4-DOF arm. The robot is operated either through a PS4 controller in manual mode or autonomously in mission mode, with a web dashboard providing live telemetry throughout.

## 11.1 Pre-Operation Checklist

Before every operating session, the following items must be verified:

- Wheels are tight and arm joints move freely.
- Battery is charged above 50%.
- Both USB cables are connected: Raspberry Pi → Base ESP32 and Raspberry Pi → Arm ESP32.
- The camera cable is connected and the lens cap is removed.
- The PS4 controller is charged, or its USB cable is available nearby.

**Warning:** Do not plug or unplug USB cables while the motors are running.

## 11.2 Power-On Sequence

1. Turn on the main battery switch.
2. Wait 3 seconds. The arm servos should be audible and the Base ESP32 LED should blink.
3. Power on the Raspberry Pi.
4. Allow approximately 30 seconds for the Pi to complete its boot sequence.
5. Pair the PS4 controller by holding **PS** + **Share** until the light bar flashes white and then turns solid.

**Tip:** If pairing fails, connect the controller to the Pi over USB first, then disconnect it once paired.

## 11.3 Software Startup

On the Raspberry Pi (locally or over SSH), run:

```
cd ~/pi_robot
python main.py --mode manual
```

A successful startup produces the following indications:

- `BASE READY` and `ARM READY` messages in the terminal.
- The dashboard becomes available at `http://[pi_ip]:8080`.

**Warning:** A `SIM mode` message indicates the serial ports failed to open. Check both USB cables and restart the software.

## 11.4 Web Dashboard

The dashboard, accessed at `http://[pi_ip]:8080`, provides real-time telemetry across six panels, summarised in Table 11.1.

**Table 11.1.** Dashboard panel summary

| Panel | Description |
|---|---|
| Camera Feed | Live video with QR and colour detection |
| IMU | Roll, pitch, yaw, and accelerometer history |
| Wheel Encoders | RPM readout for all four wheels |
| Battery | Charge level and temperature |
| Activity Log | Real-time events and status messages |
| System Mode | Indicates MANUAL, AUTO, or NAVIGATE |

**Tip:** The dashboard refreshes every half second. Use it to confirm system health before driving.


**Figure 11.1.** Web dashboard at port 8080 showing live telemetry.

## 11.5 PS4 Controller Mapping

Pressing **R3** (the right stick) toggles the controller between BASE mode and ARM mode.

## Base Mode

**Table 11.2.** PS4 controller mapping — BASE mode

| Input | Function |
|---|---|
| Left stick up/down | Drive forward / backward |
| Left stick left/right | Rotate left / right |
| Right stick up/down | Strafe left / right |
| D-pad | Drive or rotate while held |
| Triangle / X / Circle / Square | Diagonal strafe (hold) |
| R1 / L1 | Increase / decrease speed |
| R2 / L2 | Increase / decrease watchdog timeout |
| Options | Toggle MANUAL / AUTO mode |
| Share | Emergency stop and send arm home |

## Arm Mode

**Table 11.3.** PS4 controller mapping — ARM mode

| Input | Function |
|---|---|
| Left stick | Joint 0 (left/right) and Joint 1 (up/down) |
| Right stick | Joint 2 (up/down) and Joint 3 (left/right) |
| D-pad up/down | Open / close gripper |
| R2 (light press) | Fine gripper control |
| L1 | Move arm to HOME position |
| R1 | Run the built-in pick sequence |
| L3 (left stick press) | Run the Pi-side arm sequence |

**Tip:** Holding the D-pad or a diagonal-strafe button repeats the command automatically, so the robot continues moving without re-pressing.

**Figure 11.2.** PS4 controller used for manual teleoperation.

## 11.6 Calibration Procedure

Calibration is performed once at the start of every session:

1. Switch to ARM mode and press L1; the arm should move to the HOME position.
2. Use the D-pad to fully open the gripper and note the angle shown on the dashboard.
3. Switch to BASE mode and release all sticks; wheel RPM should read near zero.
4. Check the dashboard camera feed; QR codes should be sharp at 30–50 cm.
5. Place the robot on flat ground; IMU roll and pitch should read below 2°.

## 11.7 Mission Execution (AUTO Mode)

1. Place the red, green, and blue cubes in their starting positions.
2. Confirm the dashboard camera feed shows the QR codes.
3. Press **Options** on the controller to enter AUTO mode.
4. The robot executes the sequence automatically: red cube first, then green, then blue.
5. Monitor the dashboard; the state field cycles through values such as `AUTO_X_GO` and

`AUTO_Y_GO`.

6. When the state reads `AUTO_DONE`, press **Options** again to return to MANUAL mode.

**Warning:** In AUTO mode, controller inputs are ignored except Options and Share. Stay clear of the arm during autonomous operation.

## 11.8 Emergency Stop Procedures

**Table 11.4.** Emergency response actions

| Condition | Action |
|---|---|
| Robot moving on its own | Press Share on the controller |
| Arm colliding with an obstacle | Press Share (stops and homes the arm) |
| Software freeze | Disconnect the Base ESP32 USB cable |
| Smoke or fire | Trigger the main battery disconnect switch |

**Warning:** The motors stop automatically if no command is received within 600 ms. This watchdog behaviour serves as a backup safety mechanism.

## 11.9 Shutdown Sequence

1. Press L1 to send the arm to the HOME position.
2. Release all controller sticks and confirm RPM reads zero.
3. Press `Ctrl+C` in the terminal running the control software.
4. Wait for the `Shutdown complete` message in the logs.
5. Run `sudo shutdown now` on the Raspberry Pi.
6. Wait 10 seconds for the Pi's green LED to stop blinking.

7. Turn off the main battery.

**Warning:** Never disconnect power directly. The Raspberry Pi requires a proper shutdown, or the SD card may become corrupted.

## 11.10 Battery Charging Guidelines

- Use only the charger supplied with the robot.
- Charge on a non-flammable surface, ideally inside a fire-safe bag.
- Do not leave the battery charging unattended.
- For storage longer than one week, charge to approximately 50% (storage charge).

## 11.11 Troubleshooting

**Table 11.5.** Common issues and corrective actions

| Symptom | Corrective Action |
|---|---|
| `SIM mode` in logs | Disconnect and reconnect both USB cables |
| No camera image | Check `/dev/video0` or try camera index 1 |
| Controller not responding | Re-pair: hold PS + Share for 5 seconds |
| Motors shaking | Check encoder wiring; tune PID gains |
| Arm drifts from HOME | Recalibrate servo offsets in firmware |
| QR codes not detected | Clean the lens; check lighting conditions |
| Dashboard not loading | Check firewall settings; confirm port 8080 is free |
| IMU running hot | Allow it to cool; check for shorted motor wiring |

## 11.12 Summary

The operating procedure follows a consistent sequence: pre-operation checks, power-on, software startup, calibration, and either manual teleoperation or autonomous mission execution, followed by an orderly shutdown. The watchdog timeout, Share-button emergency stop, and battery disconnect switch provide layered safety coverage throughout operation, mirroring the protection philosophy applied at the hardware level in the electrical design (Chapter 6).

Key operational safeguards are summarised below:

**Table 11.6.** Operational safeguards and rationale

| Safeguard | Rationale |
|---|---|
| 600 ms command watchdog | Stops motors automatically if communication is lost |
| Share-button emergency stop | Provides an immediate manual override at all times |
| AUTO-mode input lockout | Prevents accidental interference during autonomous runs |
| Mandatory pre-operation checklist | Catches mechanical and connection faults before power-up |
| Proper Pi shutdown procedure | Prevents SD card corruption from abrupt power loss |

# Chapter 12

# Project Milestones and Phases

This chapter records the major milestones reached across the five development phases of the **pi_robot** project, mechanical design, electrical design, manufacturing, control systems, and software, along with the key design changes made at each milestone and the lessons learned along the way. A final section lists the cross-phase integration checkpoints that gated progress between phases.

## 12.1 Phase 1: Mechanical Design

*Focus: robot chassis, arm linkage, gripper mechanism, and structural analysis.*

**Table 12.1.** Phase 1 milestones — Mechanical Design

| Milestone | Major Changes | Lessons Learned |
|---|---|---|
| Concept sketch and envelope check | — | — |
| CAD model: base chassis | • Switched from a single deck to a two-layer chassis to separate battery from electronics<br>• Widened wheel base for better stability with mecanum wheels | • Leave more space for wiring |
| CAD model: arm linkage (4-DOF) | • Reduced link lengths to lower torque needed at the base joint<br>• Added cable channels along the links to hide wiring | • Longer links look nice but increase motor load a lot |
| CAD model: gripper and end-effector | • Making the gripper more configurable<br>• Adding padding foam so the gripper servo doesn't draw a lot of current<br>• Made the overall design simpler | • Soft padding reduces slipping without needing a stronger (heavier) servo |
| Design review: mechanical freeze | • Considering the wire lines so cables don't cross moving joints<br>• Considering vents for airflow around the motor drivers | • Easier to plan wire routing before parts are printed than after |

## 12.2 Phase 2: Electrical Design

*Focus: power distribution, motor drivers, sensor integration, PCB layout, and wiring harness. See Chapter ?? for the resulting board-level design.*

**Table 12.2.** Phase 2 milestones — Electrical Design

| Milestone | Major Changes | Lessons Learned |
|---|---|---|
| Power budget and component selection | • Choosing ESP32 for low-level control (arm) and ESP32-S3 for the base, so the Pi only handles high-level tasks<br>• Choosing a low-budget battery that provides the desired power for the robot | • Splitting control between two ESP32s reduced load on the Pi and made debugging easier |
| Power sizing | • Choosing the desired capacity of each battery pack (separate packs for motors vs. logic) | • Separating motor and logic power stopped the Pi from resetting under motor load |
| PCB layout: base controller board | • Routed motor driver traces thicker to handle stall current<br>• Grouped connectors by side to match wheel positions | • Labeling connectors clearly on the board saved a lot of time during assembly |
| PCB layout: arm controller board | • Grouped all servo headers on one edge of the board to keep wiring to the arm simple<br>• Added decoupling capacitors near each servo header after seeing voltage dips during movement | • Servo jitter dropped a lot after separating signal and power traces |
| Wiring harness design and connector map | • Used colour-coded wires and a simple connector map sheet | • A written connector map made re-wiring after a redesign much faster |

## 12.3 Phase 3: Manufacturing

*Focus: 3D printing, PCB fabrication, component procurement, and mechanical assembly.*

**Table 12.3.** Phase 3 milestones — Manufacturing

| Milestone | Major Changes | Lessons Learned |
|---|---|---|
| Material selection | • Chose PLA for most body parts (cheap, easy to print) | • PLA is fine for structure but gets brittle at high-stress contact points |
| 3D print: base chassis and wheel mounts | • Increased infill on wheel mounts after first print cracked under load | • Higher infill matters more than wall thickness for parts that take repeated shock |
| 3D print: arm links and gripper jaws | • Printed links with the layer lines aligned along the load direction | • Print orientation changes strength a lot, even with the same material and settings |
| PCB fabrication and assembly | • Ordered boards externally, soldered components by hand | • Double-checking footprints before ordering avoided a costly reorder |
| Mechanical assembly: base + wheels | • Added threaded inserts instead of screwing directly into printed plastic | • Threaded inserts saved the chassis from stripped screw holes after repeated assembly |
| Mechanical assembly: arm + gripper | • Assembled arm joints with washers to reduce friction between printed parts | • Small washers made a noticeable difference in how smoothly joints moved |
| Wiring assembly | • Followed the connector map from Phase 2 and added cable ties at joints | • Loose wires near moving joints can fray quickly without strain relief |

## 12.4 Phase 4: Control Systems

*Focus: PID velocity control, EKF odometry, trajectory generation, inverse kinematics, and safety interlocks.*

**Table 12.4.** Phase 4 milestones — Control Systems

| Milestone | Major Changes | Lessons Learned |
|---|---|---|
| Mecanum kinematics derivation | • Verified wheel velocity equations in simulation before testing on the real base | • Wheel roller orientation matters a lot; a single mixed-up wheel breaks all motion |
| PID controller design and simulation | • Tuned gains in simulation first, then fine-tuned on hardware | • Simulation gains were a good starting point but still needed retuning on real motors |
| Encoder odometry implementation | • Added simple low-pass filtering on raw encoder counts | • Encoder noise alone made odometry drift faster than expected |
| Quintic trajectory generator | • Used quintic profiles instead of trapezoidal for smoother arm motion | • Smoother profiles reduced mechanical jerk and servo strain noticeably |
| Inverse kinematics solver | • Added joint limit checks inside the solver to avoid invalid arm poses | • Without limit checks, the solver sometimes returned solutions the arm physically couldn't reach |
| Safety interlocks and watchdog timer | • Added a watchdog timer on both ESP32 boards to stop motors if the Pi stops responding | • A simple watchdog prevented the robot from running away during a software crash |

## 12.5 Phase 5: Software

*Focus: serial communication, computer vision, state machine, dashboard, and autonomous logic.*

**Table 12.5.** Phase 5 milestones — Software

| Milestone | Deliverables | Major Changes |
|---|---|---|
| System workflow design | • Flowchart of overall robot behaviour (Pi as brain, two ESP32s as low-level controllers) | • Split the workflow early into perception, decision, and action stages |
| Serial communication: Pi ↔ ESP32 | • Working UART link between Pi and both ESP32 boards with a simple message format | • Switched to a fixed-length packet format after losing bytes with plain text messages |
| Camera driver and OpenCV pipeline | • Basic capture and preprocessing pipeline (resize, blur, threshold) running on the Pi | • Lowered camera resolution to keep frame rate usable on the Pi |
| QR detection (pyzbar) and colour classification | • Working QR code reader and basic HSV-based colour classifier | • Switched from RGB to HSV thresholds since colour detection was unreliable under changing light |
| Decision node: state machine implementation | • State machine handling idle, search, approach, grab, and deliver states | • Added an explicit error/recovery state after the robot got stuck with no fallback |
| Dashboard (PyQt) and PS4 teleop node | • PyQt dashboard showing live camera feed and robot status, plus manual PS4 controller mode | • Manual teleop mode was essential for testing hardware before autonomy was ready |

## 12.6 Cross-Phase Integration Milestones

The checkpoints in Table 12.6 gated progress between phases, ensuring that each subsystem was verified before the next phase depended on it, consistent with the bottom-up integration sequence described in Chapter 13.

**Table 12.6.** Key integration checkpoints across all phases

| Checkpoint | Deliverables | Decision |
|---|---|---|
| Mechanical design review | • Frozen CAD model of chassis, arm, and gripper | • Approved for printing after wiring and airflow review |
| Electrical design review | • Finalised schematics for base and arm controller boards | • Approved for PCB ordering after power budget check |
| PCB ordering gate | • Final Gerber files for both boards | • Ordered both boards together to save on shipping/lead time |
| Mechanical assembly complete | • Fully assembled chassis, arm, and gripper (no electronics yet) | • Cleared to begin wiring and electronics integration |
| First power-on (smoke test) | • Verified safe voltages on all boards with no load | • No major issues found; cleared to connect motors and sensors |
| Closed-loop base control | • Base drives a commanded path using PID + odometry | • Accepted after path-following error stayed within target tolerance |
| Arm trajectory + IK working | • Arm reaches commanded (x, y, z) targets smoothly | • Accepted after repeated reach tests showed consistent, repeatable positioning |
| Vision pipeline online | • Live QR and colour detection feeding the decision node | • Accepted after detection stayed reliable across different lighting tests |
| Full autonomous demo | • End-to-end run: detect, navigate, grab, and deliver without manual input | • Marked project complete; remaining issues logged as future work |

## 12.7 Summary

The five-phase structure, mechanical, electrical, manufacturing, control systems, and software, allowed each domain to progress with a clear scope while remaining synchronised through the cross-phase integration checkpoints in Table 12.6. Recurring lessons across

phases included the value of separating motor and logic power early (Phase 2), validating models in simulation before committing to hardware (Phase 4), and keeping a written connector map and message-format discipline to make rework cheap when designs changed (Phases 2 and 5).

Key cross-phase lessons are summarised below:

**Table 12.7.** Cross-phase lessons and rationale

| Lesson | Rationale |
|---|---|
| Separate motor and logic power early | Prevented the Pi from resetting under motor load once electronics were integrated |
| Simulate before tuning on hardware | Gave PID and kinematics a reliable starting point, reducing hardware tuning time |
| Maintain a written connector map | Made re-wiring after redesigns significantly faster during manufacturing |
| Freeze designs only after a cross-discipline review | Caught wiring and airflow issues before parts were printed or boards were ordered |
| Build manual teleoperation before autonomy | Allowed hardware validation independent of vision and decision-making readiness |

# Chapter 13

# System Integration

This chapter describes how the electrical (Chapter **??**), mechanical, and software subsystems of the **pi_robot** platform are brought together into a single functioning system. It covers the integration architecture, the interface definitions between modules, the communication protocol, the autonomous state machine, calibration procedures, the pre-competition integration checklist, the safety interlocks and risk assessment, and the integration timeline followed during development.

## 13.1 Integration Architecture

The robot uses a three-layer hierarchy. The Raspberry Pi 4 coordinates perception and planning. Two ESP32 microcontrollers handle real-time motor control. Actuators and sensors form the physical layer. All inter-processor communication uses USB serial at 115200 baud.

**Table 13.1.** Integration layer responsibilities

| Layer | Hardware | Responsibility |
|---|---|---|
| High-Level | Raspberry Pi 4 | Vision, decision-making, HMI, logging |
| Mid-Level | ESP32-S3 (Base), ESP32 (Arm) | Real-time motor control, sensor reading |
| Low-Level | Motors, servos, sensors | Physical actuation and sensing |

The star topology keeps the Pi as the sole coordinator. The Base and Arm ESP32s never communicate directly; all commands flow through the Pi. This simplifies debugging and ensures the Pi has full system state visibility, consistent with the communication architecture described in Section **??**.

## 13.2 Interface Definitions

Every module pair has a defined interface with physical connections, data format, and timing.

### Raspberry Pi to Base ESP32

**Table 13.2.** Pi–Base ESP32 interface

| Parameter | Value |
|---|---|
| Physical connection | USB cable (UART over USB) |
| Port on Pi | `/dev/ttyACM0` |
| Baud rate | 115200 |
| Data format | ASCII text, newline-terminated |
| Update rate | 50 Hz (20 ms period) |
| Command types | Motion, speed, PID tuning, stop |
| Feedback types | Encoder ticks, IMU yaw, odometry, status |
| Watchdog timeout | 600 ms |

## Raspberry Pi to Arm ESP32

**Table 13.3.** Pi–Arm ESP32 interface

| Parameter | Value |
|---|---|
| Physical connection | USB cable (UART over USB) |
| Port on Pi | `/dev/ttyUSB0` |
| Baud rate | 115200 |
| Data format | ASCII text or compact frame (`$j0,j1,j2,j3,grip`) |
| Update rate | 50 Hz command, 100 Hz trajectory |
| Command types | Joint angles, IK target, gripper, home, stop |
| Feedback types | Joint angles, Cartesian position, moving flag |
| Trajectory type | Quintic polynomial, 200–3000 ms |

## Camera to Raspberry Pi

**Table 13.4.** Camera–Pi interface

| Parameter | Value |
|---|---|
| Physical connection | CSI-2 ribbon (Pi Camera) or USB (webcam) |
| Resolution | 640 × 480 pixels |
| Frame rate | 30 FPS |
| Colour format | RGB24 |
| Lens | Wide-angle (160° diagonal) |
| Working distance | 15–60 cm for QR detection |
| Software library | OpenCV + picamera2 |

### Mechanical to Electrical

**Table 13.5.** Mechanical–electrical mounting interfaces

| Component | Mounting | Electrical Connection |
|---|---|---|
| Base PCB | M3 standoffs on chassis floor | 12 V battery, 4 motor JST pairs, IMU I2C |
| Arm PCB | M3 standoffs on shoulder link | 12 V battery, 5 servo headers, Pi USB |
| Raspberry Pi | DIN rail clip on mid-frame | USB to Base, USB to Arm, CSI camera, Wi-Fi |
| Camera | Adjustable bracket on front mast | CSI-2 ribbon to Pi |
| Battery packs | Velcro straps in lower bay | XT60 connectors to both PCBs |

## 13.3 Communication Protocol

The serial protocol uses human-readable ASCII. Every command receives an acknowledgement.

### Command Format

$$\texttt{DEVICE:COMMAND:VALUE1:VALUE2}\backslash n \tag{13.1}$$

The device prefix routes to the correct ESP32. The Base ESP32 accepts `BASE:` prefixes. The Arm ESP32 accepts bare commands (`HOME`, `JOINT`) or `ARM:` prefixes.

## Base Command Set

**Table 13.6.** Base ESP32 commands

| Command | Effect |
|---|---|
| `BASE:FORWARD` | All wheels forward at current speed |
| `BASE:BACKWARD` | All wheels reverse at current speed |
| `BASE:LEFT` | Strafe left (mecanum diagonal) |
| `BASE:RIGHT` | Strafe right (mecanum diagonal) |
| `BASE:ROTATECW` | Rotate clockwise in place |
| `BASE:ROTATECCW` | Rotate counter-clockwise in place |
| `BASE:STOP` | Zero all motor PWM, hold position |
| `BASE:SPEED:<n>` | Set global speed, $n \in [0, 255]$ |
| `BASE:VEL:<s>:<d>` | Combined speed $s$ and direction $d$ |
| `BASE:TURN:<s>:<d>` | Combined speed $s$ and turn $d$ |
| `BASE:PID:<kp>,<ki>,<kd` | Live PID gain update |

## Arm Command Set

**Table 13.7.** Arm ESP32 commands

| Command | Effect |
|---|---|
| `HOME` | Move all joints to zero, gripper to 90° |
| `STOP` | Freeze at current position |
| `JOINT <j> <deg>` | Move joint $j$ to absolute angle |
| `IK <x> <y> <z>` | Solve IK and move to Cartesian target |
| `GRIPPER <deg>` | Set gripper angle, $[0, 180]$ |
| `SPEED <ms>` | Set motion duration in milliseconds |
| `STATE` | Request current joint state |
| `$j0,j1,j2,j3,grip` | Compact 5-axis frame (joystick mode) |

### Feedback Format

Base ESP32 sends:

$$\texttt{ODOM:}\omega_{FL},\omega_{FR},\omega_{RL},\omega_{RR} \quad \texttt{IMU:}\theta_{yaw} \quad \texttt{OK:}\langle\text{cmd}\rangle \tag{13.2}$$

Arm ESP32 sends:

$$\texttt{STATE deg=}\theta_1,\theta_2,\theta_3,\theta_4 \ \texttt{xyz=}x,y,z \ \texttt{moving=}\{0,1\} \ \texttt{gripper=}g \tag{13.3}$$

## 13.4 State Machine

The Decision Node implements a finite state machine for autonomous behaviour.

**Table 13.8.** Autonomous state machine

| State | Entry Condition | Action |
|---|---|---|
| MANUAL | Mode switch or startup | Pass through joystick/keyboard commands |
| AUTO_X_GO | QR detected RED | Strafe left 2.0 s at 35% speed |
| AUTO_X_PLACE | X_GO timer expired | Lower arm, close gripper, raise arm |
| AUTO_X_BACK | PLACE done | Strafe right 2.0 s to return |
| AUTO_Y_GO | QR detected GREEN | Move forward 2.0 s |
| AUTO_Y_PLACE | Y_GO timer expired | Place sequence for GREEN bin |
| AUTO_Y_BACK | PLACE done | Move backward 2.0 s |
| AUTO_D_GO | QR detected BLUE | Diagonal forward-left 2.8 s |
| AUTO_D_PLACE | D_GO timer expired | Place sequence for BLUE bin |
| AUTO_D_BACK | PLACE done | Diagonal return 2.8 s |
| AUTO_DONE | All cubes placed | Idle, awaiting operator reset |
| PI_SEQ | Manual trigger | Execute Pi-side arm pick sequence |

Transitions trigger on: vision events (`VISION_QR_DETECTED`), timer expirations, serial `OK` acknowledgements, or manual mode toggle.

## 13.5 Calibration Procedure

### Base Calibration

1. **Encoder ticks-per-metre:** Drive 1.0 m forward, count ticks. Compute $k_{enc}$ = ticks/1.0 per wheel.
2. **Wheel diameter:** Measure actual $D_{actual}$. Update $d_{tick} = \pi D_{actual}/CPR$.
3. **Track width:** Rotate 10 turns in place. Measure actual rotation. Correct $W$ in odometry.
4. **PID tuning:** Find $K_p$ from oscillation test, add $K_i$ for steady-state removal.
5. **IMU bias:** Leave stationary 5 s. Average gyro readings. Subtract bias in software.

### Arm Calibration

1. **Servo zero:** Command 1500 $\mu$s pulse. Adjust horn until link is horizontal.
2. **Joint limits:** Move to $\pm 90°$. Confirm no mechanical interference.
3. **Link lengths:** Measure $L_1$ through $L_4$ with calipers. Update IK constants.
4. **Gripper zero:** Close at 0°, open at 180°. Verify cube fits at 105°.
5. **Trajectory timing:** Time HOME to PICK motion. Adjust duration for smooth motion.

### Camera Calibration

1. **Lens distortion:** Capture checkerboard. Compute OpenCV `calibrateCamera` coefficients.
2. **Pixel-to-distance:** Place cube at known distances. Fit $z = f(\text{area})$ curve.
3. **Colour thresholds:** Sample cubes under arena lighting. Record HSV bounds.
4. **QR minimum area:** Find smallest detectable QR. Set `min_qr_area` with 20% margin.

## 13.6 Integration Checklist

**Table 13.9.** Pre-competition integration checklist

| # | Test | Pass Criteria | Status |
|---|---|---|---|
| 1 | Power-on self-test | All LEDs on, no smoke | Pass |
| 2 | Serial loopback | Echo command to ESP32 and back | Pass |
| 3 | Base motor spin | Each wheel forward and reverse | Pass |
| 4 | Base strafe test | Robot moves left, right, diagonal | Pass |
| 5 | Encoder feedback | ODOM values change with motion | Pass |
| 6 | IMU response | Yaw changes during rotation | Pass |
| 7 | Arm home position | All joints reach 0° | Pass |
| 8 | Arm pick sequence | Full pick-and-place without jamming | Pass |
| 9 | Gripper force hold | Cube held 10 s without dropping | Pass |
| 10 | Camera stream | 30 FPS, no tearing, correct colours | Pass |
| 11 | QR detection | Decodes all 3 cube colours reliably | Pass |
| 12 | Autonomous sequence | Full RED pick–place cycle completes | Pass |
| 13 | PS4 pairing | Controller connects within 10 s | Pass |

## 13.7 Safety Interlocks and Risk Assessment

The safety system combines proactive interlocks (preventing dangerous conditions) with reactive mitigations (responding to faults). Table 13.10 presents the FMEA-style risk assessment, mapping each identified hazard to its cause, effect, severity, likelihood, and mitigation strategy. Table 13.11 then maps these risks to the specific hardware and software interlocks implemented.

## FMEA-Style Risk Assessment

**Table 13.10.** Risk assessment (FMEA-style)

| Risk | Cause | Effect | Sev | Lik | RPN | Mitigation |
|---|---|---|---|---|---|---|
| Robot exceeds 10 kg | Heavy arm / battery selection | DQ from competition | 5 | 2 | 10 | Use CFRPLA, LiPo 4S; weigh after each build phase |
| QR code not read autonomously | Poor lighting / camera angle | No autonomous pick | 4 | 3 | 12 | Test QR at various angles; add LED illumination ring |
| Omni wheel slippage | Floor texture variation | Odometry drift | 3 | 4 | 12 | IMU heading fusion; recalibrate before each run |
| Arm drops cube during transit | Gripper force too low | −10 penalty; time loss | 4 | 2 | 8 | Compliant finger pads; close-loop gripper force control |
| USB serial communication loss | Loose USB cable / connector wear | Module stops, no control | 5 | 2 | 10 | Heartbeat watchdog; hardware E-Stop fallback; cable strain relief |
| Battery over-discharge | Long run time / current spike | Shutdown mid-run | 4 | 2 | 8 | BMS with low-voltage cutoff; 20% capacity reserve rule |
| PCB short circuit | Exposed terminal / assembly error | Fire / system failure | 5 | 1 | 5 | All PCBs in enclosure; fuse on each rail; TVS diodes |
| Arm collision with robot body | IK singularity / no joint limits | Mechanical damage | 3 | 3 | 9 | Software joint limit checks + hardware end-stops |

**RPN** = Risk Priority Number = Severity × Likelihood. Risks with RPN ≥ 10 require mandatory mitigation before competition.

## Safety Interlock Implementation

**Table 13.11.** Safety interlock summary

| Interlock | Trigger | Response | Risk Addressed |
|---|---|---|---|
| Software watchdog | 600 ms no command | Zero all motor PWM | USB serial loss, Pi crash |
| E-Stop button | Physical press | Cut 12 V main power | PCB short, fire, collision |
| Joint limits | Angle out of range | Clamp to nearest limit | Arm collision, IK singularity |
| Serial parse error | Invalid command | Send ERR, ignore command | USB communication noise |
| Overcurrent | Fuse blow | Open circuit, replace fuse | PCB short, motor stall |
| Low battery | Voltage < 11.0 V | LED warning, reduce speed | Battery over-discharge |
| Weight audit | Scale measurement | Reject if > 10 kg | Robot exceeds 10 kg |
| QR illumination | Ambient light < threshold | Auto-enable LED ring | QR not read |
| IMU recalibration | Pre-run check | Zero gyro bias, reset EKF | Omni wheel slippage |
| Gripper force pad | Mechanical compliance | TPU fingers absorb ±2 mm | Cube drop during transit |
| USB strain relief | Cable tie / routing | Prevent connector fatigue | USB serial communication loss |

## Software Watchdog

The Base ESP32 monitors command arrival time. If no valid command arrives for 600 ms, all PWM outputs go to zero.

$$\text{if } (t_{now} - t_{last\_cmd}) > 600 \text{ ms} \Rightarrow \text{PWM}_{1..4} = 0 \tag{13.4}$$

The Pi sends heartbeat messages at 20 Hz. If the Pi crashes or USB disconnects, the base stops automatically. This addresses the USB serial communication loss risk (RPN = 10).

## E-Stop Circuit

A latching normally-closed emergency stop button breaks the main 12 V supply. Pressing it immediately removes power from all motors and servos. The button must be twisted to reset. This is the ultimate fallback for PCB short circuit (RPN = 5) and arm collision risks.

## Joint Limit Enforcement

The Arm ESP32 clamps every commanded angle to $[-90°, +90°]$ before sending to servos.

$$\theta_{safe} = \max(-90°, \min(+90°, \theta_{cmd})) \tag{13.5}$$

Hardware endstops on joints 2 and 3 provide mechanical backup. This dual protection prevents arm collision with the robot body (RPN = 9).

## Battery Protection

The Battery Management System (BMS) monitors cell voltage and cuts discharge below 3.0 V per cell. A 20% capacity reserve rule ensures the robot never enters the competition with less than 20% charge remaining.

$$\text{usable\_capacity} = 0.8 \times \text{nominal\_Ah} \tag{13.6}$$

This mitigates battery over-discharge (RPN = 8).

### USB Serial Reliability

USB cables are secured with cable ties and routed through strain-relief channels to prevent connector fatigue. The Pi uses `/dev/ttyACM0` and `/dev/ttyUSB0` with automatic port detection. If a port disappears, the Serial Node logs the error and retries every 2 seconds.

$$\text{reconnect\_interval} = 2.0 \text{ s} \tag{13.7}$$

This addresses USB serial communication loss (RPN = 10) by reducing mechanical stress on the connectors.

### Communication Error Handling

Invalid commands are rejected with an `ERR` response. The parser resets on newline, so corrupted byte streams recover at the next valid command boundary. This handles USB communication noise by preventing parser lockup.

## 13.8 Integration Timeline

Integration followed a bottom-up sequence: electrical first, then mechanical mounting, then software deployment, and finally system testing.

**Table 13.12.** Integration milestone schedule

| Week | Milestone | Deliverable |
|---|---|---|
| 1 | PCB power-on | Both boards boot, LEDs correct |
| 2 | Motor spin | Base wheels spin, arm servos respond |
| 3 | Sensor check | Encoders, IMU, camera all streaming |
| 4 | Closed-loop base | PID velocity control working |
| 5 | Arm trajectory | Quintic motion smooth, IK converging |
| 6 | Pi–ESP32 link | Serial commands round-trip reliably |
| 7 | Vision online | QR detection, colour classification |
| 8 | Autonomous demo | Full pick–place cycle unattended |
| 9 | HMI complete | PS4, keyboard, dashboard all functional |
| 10 | Competition ready | All checklist items passed |

The parallel development structure meant software was tested in simulation while hardware was built. When the PCBs (Chapter **??**) arrived, the Pi-side code deployed immediately without fundamental changes.

## 13.9 Summary

System integration followed a deliberate bottom-up sequence, layering the electrical, mechanical, and software subsystems through clearly defined interfaces and a single serial protocol. The star communication topology, the autonomous state machine, and the layered safety interlocks together ensured that every subsystem boundary identified in this chapter was tested before the platform reached the full-system trials described in Chapter **??**.

Key integration decisions are summarised below:

**Table 13.13.** Integration decisions and rationale

| Decision | Rationale |
|---|---|
| ASCII serial protocol with acknowledgement | Simple to debug, robust to partial data loss, easy to log |
| Centralised finite state machine on the Pi | Single source of truth for autonomous behaviour, simplifies transitions |
| 600 ms watchdog + 20 Hz heartbeat | Detects USB/Pi failures quickly without false-triggering on normal latency |
| FMEA-driven safety interlocks | Prioritises mitigation effort on the highest RPN risks first |
| Bottom-up integration timeline | Validates each layer before the next depends on it, reducing rework |

# Chapter 14

# Technical Challenges and Lessons Learned

This chapter lists the main problems encountered during development of the **pi_robot** platform, why they happened, and how they were resolved. Each problem is traced from cause through fix to the lesson that would change how it is approached next time.

## 14.1 Serial Communication

**Problem:** The Base ESP32 reset every time the Python serial port opened. The motors twitched and the robot froze.

**Cause:** The USB-to-UART chip on the ESP32 board uses the DTR and RTS lines to control reset and boot mode. Python turns these on by default when opening the port, so the ESP32 rebooted unexpectedly.

**Fix:** DTR and RTS were disabled in the `SerialNode` code before setting the baud rate. Auto-reconnect was also added: if the port fails, the node retries every 2 seconds instead of crashing.

**Next time:** Use the Pi's GPIO UART pins instead of USB. This removes the reset problem and cuts delay by about 5 ms.

## 14.2 Power Noise

**Problem:** When the arm servos moved, the Raspberry Pi rebooted or lost USB connections. The camera and dashboard died.

**Cause:** The servos and the Pi shared one 5 V rail. The servos draw up to 2.5 A in stall, causing a voltage drop that browns out the Pi.

**Fix:** The power was split into two rails on the Arm PCB, an XL4016 buck converter for the servos and an LM2596 for the Pi and ESP32 logic, matching the power-isolation strategy described in Section **??**. 1000 µF capacitors were also added to absorb power spikes.

**Delay:** One week to redesign and remake the PCB. A breadboard workaround was used to keep testing software during that week.

**Next time:** Add a supercapacitor on the logic rail to survive short power dips, and a voltage alarm when the servo rail drops below 11 V.

## 14.3 Wheel Contact and Direction Deviation

**Problem:** The robot did not always drive in a straight line. Over a 1 m run it could pull noticeably to one side.

**Cause:** Poor mechanical tolerances in the frame meant the wheels were not always fully touching the ground at the same time. With uneven contact, one or more wheels lost traction briefly, so the robot did not move evenly on all sides.

**Fix:** The wheel mounts were re-checked and shimmed so all four wheels sat at the same height. Small spacers were added where the frame was slightly bent, and the wheel axle bolts that had worked loose were retightened.

**Next time:** Design the chassis with a small amount of suspension or spring-loaded wheel mounts so all wheels stay in contact with the ground even on slightly uneven surfaces.

## 14.4 Unstable Arm Base

**Problem:** The base of the arm was not stable. It wobbled when the arm moved and leaned to one side, like a leaning tower.

**Cause:** The arm base was mounted with too few screws on a thin bracket, so it could flex under the arm's weight and motion.

**Fix:** A large bushing was designed and 3D-printed to mount on the arm base. It fits around the pivot point and takes up the play between the moving arm and the base, which removed the wobble and stopped the lean.

**Next time:** Design the bushing into the base from the start, sized and tested early, instead of adding it as a later fix.

## 14.5 Gripper Servo Overload

**Problem:** The gripper servo would overload and eventually burn out during repeated use.

**Cause:** The gripper fingers were rigid PLA, a material with low ductility. Because the fingers could not flex at all, the servo had to push against the full resistance of the cube with no give, drawing high current and overheating until it failed.

**Fix:** A layer of foam was added to the gripper fingers. This let the gripper press and hold the cube with a small amount of give, so the servo did not have to fight against a completely rigid surface, reducing the load on the servo and stopping the overheating.

**Next time:** Use a slightly flexible fingertip material (such as TPU) from the start, and add a current check that stops the servo from pushing once it senses it has gripped the object.

## 14.6 Vision Failures

**Problem:** QR detection worked in the lab but failed in the competition arena. Glare and shadows blocked the codes.

**Cause:** OpenCV's QR detector needs even lighting. The arena lights created bright spots and dark shadows on the cubes. The camera auto-exposure also washed out the codes.

The HSV colour thresholds were tuned to lab lights and failed under the arena's warmer lights.

**Fix:** A ring of 12 white LEDs was added around the camera for steady lighting, the camera exposure was locked to manual mode, the HSV colour thresholds were widened by 15%, and a white-balance step was added at startup.

**Next time:** Use a k-NN colour classifier that learns the arena lighting in 30 seconds instead of fixed HSV thresholds.

## 14.7 Summary

Table 14.1 consolidates the six problems described above, mapping each to its root cause, its effect on the system, and the fix that resolved it.

**Table 14.1.** Problem summary

| Problem | Cause | Effect | Fix |
|---|---|---|---|
| ESP32 resets | DTR/RTS lines | Motors twitch | Disable DTR/RTS in code |
| Pi reboots | Shared 5 V rail | Camera dies | Two separate buck converters |
| Direction deviation | Uneven wheel contact | Robot pulls off line | Shimmed wheel mounts |
| Wobbly arm base | Weak base mounting | Arm leans, unstable | 3D-printed bushing on base |
| Gripper servo burnout | Rigid PLA fingers | Servo overheats, fails | Foam padding on fingers |
| QR missed | Glare, bad exposure | No auto pick | LED ring, manual exposure |

## 14.8 Future Improvements

1. Use Pi GPIO UART instead of USB to avoid DTR/RTS resets.
2. Add a supercapacitor on the 5 V rail to survive servo spikes.

3. Add spring-loaded or suspended wheel mounts for consistent ground contact.
4. Build the arm base bushing into the design from the start instead of as a later fix.
5. Replace HSV with an on-site k-NN colour learner.
6. Use flexible fingertip material and current sensing on the gripper from the start.

*The main lesson: lab testing is not enough. Test under real competition lighting, flooring, and time pressure before the final week.*